Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

13 RNN과 CNN을 이용한 텍스트 분류

지금까지 배운 전처리(09장), 임베딩(12장), RNN(07장), CNN(08장)을 하나로 이어 영화 리뷰가 긍정인지 부정인지 맞히는 모델을 만듭니다. 실제 리뷰 수만 개로 학습시키고, 모델이 어디서 틀리는지도 확인합니다.

오늘 다루는 것

1. RNN을 이용한 텍스트 분류: 다 대 일 구조
2. LSTM을 이용한 네이버 영화 리뷰 분류
3. GRU를 이용한 IMDB 리뷰 분류
4. 1D CNN을 이용한 IMDB 리뷰 분류
5. 사전 훈련된 임베딩을 이용한 성능 상승

끝나고 나면 할 수 있는 것

1. 리뷰 한 줄이 정수 텐서, 임베딩, RNN(또는 CNN), 출력층을 지나 "긍정/부정"이 되기까지 텐서의 모양을 따라갈 수 있다.
2. 학습, 검증, 테스트 데이터의 역할과 체크포인트를 저장하는 이유를 설명할 수 있다.
3. 1D 합성곱과 맥스 풀링이 문장에서 무엇을 계산하는지 설명할 수 있다.

2026년 10월 13일 화요일 14:30

코랩에서는 `!pip install konlpy gensim` 을 먼저 실행합니다. 실습 2~10은 **번호 순서대로** 실행하고(앞 실습이 저장한 파일을 씁니다), 실습마다 1~10분이 걸립니다.

13

RNN과 CNN을 이용한 텍스트 분류

"이 영화 정말 재밌어요"는 긍정, "시간 아까운 영화"는 부정입니다. 사람에게는 쉬운 이 판단을 모델이 하게 만드는 것이 텍스트 분류입니다. 스팸 메일 걸러 내기, 리뷰의 감정 파악, 뉴스 주제 나누기가 모두 같은 문제입니다. 12장까지는 부품을 하나씩 배웠고, 13장에서는 그 부품들을 하나의 모델로 조립해 실제 데이터로 학습시킵니다.

왜 배우나: 오늘의 학습 경로



다섯 소챕터가 모두 이 경로를 따릅니다. 달라지는 것은 네 번째 역뿐입니다. 문장을 벡터 하나로 요약하는 방법으로 LSTM(13-02), GRU(13-03), 1D CNN(13-04)을 차례로 쓰고, 마지막에 세 번째 역의 임베딩을 사전 훈련된 것으로 바꿉니다(13-05). 개념마다 정의를 먼저 세우고, 영화 리뷰의 긍정/부정 분류에 적용해 실제 숫자로 확인합니다.

13-01 RNN을 이용한 텍스트 분류

핵심 한 줄

텍스트 분류는 글을 다 읽은 뒤 정답 하나를 내는 다 대 일 문제이고, RNN의 마지막 시점의 은닉 상태를 출력층에 넣어 클래스마다의 점수를 얻는다.

왜 배우나

뒤의 네 소챕터는 코드가 길지만 뼈대는 같습니다. 여기서 그 뼈대를 먼저 세웁니다. 데이터가 어떻게 생겼는지, 왜 나눠 쓰는지, RNN의 어느 출력을 분류에 쓰는지, 출력층과 손실을 어떻게 고르는지를 작은 예로 확인해 두면 뒤의 실습을 읽기 쉽습니다.

1) 텍스트 분류와 훈련 데이터

정의

- 텍스트 분류(text classification): 글을 입력받아 미리 정해 둔 범주(클래스) 가운데 하나를 고르는 일
- 지도 학습(supervised learning): 문제와 정답이 짝지어진 데이터로 학습하는 방법. 정답을 레이블(label)이라고 합니다. 텍스트 분류는 지도 학습입니다.
- 훈련 데이터와 테스트 데이터: 가진 데이터를 전부 학습에 쓰지 않고 일부를 남겨 둡니다. 학습에 쓴 것이 훈련 데이터, 남겨 둔 것이 테스트 데이터입니다. 글의 열을 X, 레이블의 열을 y라고 부르며, `X_train, y_train, X_test, y_test` 로 나눕니다.

왜 테스트 데이터를 남겨 둘까? 모델을 만드는 목적은 처음 보는 글을 맞히는 것입니다. 학습에 쓴 글로 시험하면 외워서 맞힌 것인지 알 수 없습니다. 한 번도 보여 주지 않은 글로 정답률을 재야 실제 성능을 알 수 있습니다(06-01).

단어에 번호를 붙이는 방법. 글은 09-05의 방법으로 정수의 줄이 됩니다. 단어를 많이 나온 순서로 정렬해 번호를 붙이면, "번호가 큰 단어 = 드문 단어"가 되어 번호만 보고 드문 단어를 잘라 낼 수 있습니다.

예시 적용

스팸 메일 분류기의 훈련 데이터는 이런 모양입니다. 메일 2만 통이 있다면 18,000통으로 학습하고 2,000통은 시험용으로 남겨 둡니다.

텍스트 (메일의 내용)	레이블 (스팸 여부)
당신에게 드리는 마지막 혜택! …	스팸 메일
내일 뵐 수 있을지 확인 부탁…	정상 메일
(광고) 멋있어질 수 있는…	스팸 메일

2) RNN으로 분류하기: 다 대 일

정의

07-01에서 본 `nn.RNN(input_size, hidden_size, batch_first=True)` 를 텍스트 분류의 말로 읽으면 이렇습니다(LSTM, GRU도 같습니다).

RNN의 말	텍스트 분류에서의 뜻
시점의 수 (timesteps)	글 하나의 단어 수 L (패딩으로 맞춘 길이)
`input_size`	단어 하나를 나타내는 벡터의 크기 D (임베딩 차원)
`hidden_size`	은닉 상태의 크기 H. 글을 요약한 벡터의 크기

$$(N,\ L) \xrightarrow{\text{Embedding}} (N,\ L,\ D) \xrightarrow{\text{RNN}} \underbrace{h_L}_{(N,\ H)} \xrightarrow{\text{Linear}} \underbrace{\text{scores}}_{(N,\ K)}$$

N은 글의 수, K는 클래스의 수입니다. RNN은 모든 시점에서 은닉 상태를 내지만, 분류에는 **마지막 시점의 은닉 상태 h_L 하나만** 씁니다. 글을 끝까지 읽은 뒤의 은닉 상태가 글 전체를 요약한 것이기 때문입니다. 이것이 07-01의 다 대 일(many-to-one) 구성입니다.

출력층과 손실 함수는 클래스의 수에 따라 고릅니다.

문제	출력층의 크기	활성화 함수	파이토치 손실 함수
이진 분류 (둘 중 하나)	1	시그모이드 (04장)	`BCEWithLogitsLoss`
다중 클래스 분류 (셋 이상 중 하나)	클래스 수 K	소프트맥스 (05장)	`CrossEntropyLoss`

이진 분류도 출력을 2개로 두고 소프트맥스로 풀 수 있습니다. 교재의 실습은 모두 이 방법을 써서, 긍정/부정 문제에 출력 2개와 `CrossEntropyLoss` 를 씁니다. `CrossEntropyLoss` 는 안에서 소프트맥스를 계산하므로 모델은 소프트맥스를 거치지 않은 점수(로짓)를 내면 됩니다.

예시 적용 영어 메일 6통(스팸 3, 정상 3)으로 작은 분류기를 만듭니다(실습 1). 가장 긴 메일이 4단어이므로 $L = 4$, 임베딩 차원 $D = 8$, 은닉 크기 $H = 16$, 클래스 $K = 2$입니다.

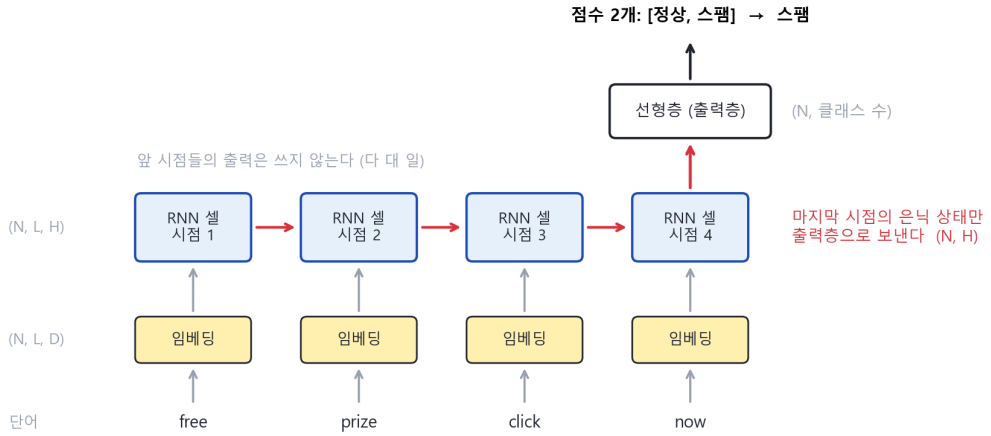


그림 1 "free prize click now"가 단어마다 임베딩을 거쳐 RNN을 지난다. 마지막 시점의 은닉 상태만 선형층으로 가서 정상과 스팸의 점수가 된다

단계	모양	뜻
입력	(6, 4)	메일 6통, 단어 4개씩 (번호)
임베딩	(6, 4, 8)	단어마다 8칸짜리 벡터
RNN의 모든 시점	(6, 4, 16)	시점마다 16칸짜리 은닉 상태
RNN의 마지막 시점	(6, 16)	메일마다 요약 벡터 하나
출력층	(6, 2)	메일마다 [정상, 스팸]의 점수

100번 학습하면 6통을 모두 맞힙니다. 학습에 없던 "free money now"는 스팸(확률 1.00), "lunch meeting tomorrow"는 정상(스팸일 확률 0.01)으로 판단합니다. 메일 6통으로는 "free, money, now가 나오면 스팸" 정도만 배운 것이고, 실제 문제에는 훨씬 많은 데이터가 필요합니다.

비유

영화를 다 보고 매기는 별점

영화 평론가는 장면마다 점수를 매기지 않습니다. 처음부터 끝까지 보면서 인상을 쌓아 가다가, 영화가 끝난 뒤에 좋다/나쁘다 하나를 정합니다.
신입 평론가는 정답(선배의 평가)이 달린 영화를 많이 보며 배우고, 실력은 한 번도 보지 않은 영화로 확인합니다.

장면을 차례로 보기 = = 단어를 시점마다 읽기
쌓여 가는 인상 = = 은닉 상태
다 본 뒤의 평가 하나 = = 마지막 은닉 상태 → 출력층 (다 대 일)
처음 보는 영화로 실력 확인 = = 테스트 데이터

실습 1

RNN으로 스팸 메일 분류하기: 다 대 일의 모양

```
import torch
import torch.nn as nn

torch.manual_seed(0)
# 텍스트(메일 내용)와 레이블(1 = 스팸, 0 = 정상)
train = [("free prize click now", 1), ("win money now", 1), ("free money offer", 1),
         ("meeting at noon tomorrow", 0), ("see you at lunch", 0),
         ("project report due tomorrow", 0)]

# 단어에 번호 붙이기 (0 = 패딩, 1 = 모르는 단어) → 정수 인코딩 → 패딩
words = sorted({w for text, _ in train for w in text.split()})
word_to_index = {"<PAD>": 0, "<UNK>": 1, **{w: i + 2 for i, w in enumerate(words)}}
max_len = max(len(text.split()) for text, _ in train)

def encode(text):
    ids = [word_to_index.get(w, 1) for w in text.split()]
    return ids + [0] * (max_len - len(ids))

X = torch.tensor([encode(text) for text, _ in train])     # (메일 수, 단어 수)
y = torch.tensor([label for _, label in train])

class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, input_size, hidden_size, num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, input_size, padding_idx=0)
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
```

```
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        e = self.embedding(x)             # (메일 수, 시점 수, input_size)
        out, h_n = self.rnn(e)            # out: 모든 시점, h_n: 마지막 시점
        return self.fc(h_n.squeeze(0))    # 마지막 은닉 상태 하나로 분류 (다 대 일)

model = RNNClassifier(len(word_to_index), input_size=8, hidden_size=16,
                      num_classes=2)
e = model.embedding(X)
out, h_n = model.rnn(e)
print("입력 X          :", tuple(X.shape), "= (메일 6통, 단어 4개)")
print("임베딩          :", tuple(e.shape), "= (메일, 시점 수, input_size)")
print("RNN 모든 시점   :", tuple(out.shape), "= (메일, 시점 수, hidden_size)")
print("RNN 마지막 시점 :", tuple(h_n.squeeze(0).shape), "← 이것만 출력층으로")
print("출력층 점수     :", tuple(model(X).shape), "= (메일, 클래스 수)")

optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
for epoch in range(100):
    loss = nn.functional.cross_entropy(model(X), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
pred = model(X).argmax(1).tolist()
print(f"학습 후 손실 {loss.item():.4f}, 예측 {pred}, 정답 {y.tolist()}")

# 학습에 없던 메일로 시험
for text in ["free money now", "lunch meeting tomorrow", "click now"]:
    prob = torch.softmax(model(torch.tensor([encode(text)])), dim=1)[0]
    label = "스팸" if prob[1] > prob[0] else "정상"
    print(f"  '{text}' → {label} (스팸일 확률 {prob[1]:.2f})")
```

실행 결과

```
입력 X          : (6, 4) = (메일 6통, 단어 4개)
임베딩          : (6, 4, 8) = (메일, 시점 수, input_size)
RNN 모든 시점   : (6, 4, 16) = (메일, 시점 수, hidden_size)
RNN 마지막 시점 : (6, 16) ← 이것만 출력층으로
출력층 점수     : (6, 2) = (메일, 클래스 수)
학습 후 손실 0.0002, 예측 [1, 1, 1, 0, 0, 0], 정답 [1, 1, 1, 0, 0, 0]
  'free money now' → 스팸 (스팸일 확률 1.00)
  'lunch meeting tomorrow' → 정상 (스팸일 확률 0.01)
  'click now' → 스팸 (스팸일 확률 1.00)
```

직접 해 보기 `hidden_size`를 16에서 32로 바꾸면 어느 줄의 모양이 바뀌나요? `num_classes`를 3으로 바꾸면(스팸/정상/광고) 어느 줄이 바뀌나요?

확인 문제

13-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 텍스트 분류는 정답(레이블)이 있는 데이터로 학습하는 지도 학습이다. (O / X) 이름 _____

2 모델의 성능은 학습에 쓴 데이터로 재는 것이 가장 정확하다. (O / X) 이름 _____

3 텍스트 분류에서 RNN의 어느 출력을 출력층에 넣나요? 이름 _____

① 첫 시점의 은닉 상태 ② 마지막 시점의 은닉 상태 ③ 입력 임베딩

4 클래스가 5개인 분류 문제에서 출력층의 크기는? 이름 _____

① 1 ② 2 ③ 5

5 텍스트 분류에서 RNN의 시점의 수는 글의 _____ 수와 같다. 이름 _____

6 실습에서 RNN의 마지막 시점 은닉 상태의 모양은 (6, _____)였다. 이름 _____

7 다 대 일 구성을 정의하고, 영화를 다 보고 매기는 별점에 빗대어 설명해 보세요. 이름 _____

원문: 13-01 RNN을 이용한 텍스트 분류 — https://wikidocs.net/66115 (교재에는 실습이 없어, 모양을 확인하는 작은 실습을 추가했습니다)

13-02 LSTM을 이용한 네이버 영화 리뷰 분류

핵심 한 줄

리뷰를 정제, 토큰화, 정수 인코딩, 패딩으로 (리뷰 수, 30) 텐서로 만들고, 임베딩 → LSTM → 선형층 모델을 학습시키되 검증 손실이 가장 낮을 때의 모델을 저장해 쓴다.

왜 배우나

처음으로 실제 데이터 수만 개를 처음부터 끝까지 다룹니다. 여기서 쓰는 순서(데이터 정제, 세 묶음으로 나누기, 단어 집합, 패딩 길이 정하기, 학습하며 검증하기, 가장 좋은 모델 저장하기)는 이후의 모든 실습에서 그대로 반복됩니다. 한 번 제대로 따라가 두면 뒤의 소챕터는 달라진 부분만 보면 됩니다.

1) 데이터 준비

정의

리뷰 —정제→ —토큰화→ —나누기→ —단어 집합→ —정수 인코딩→ —패딩→ 텐서 (N, L)

09-05에서 리뷰 6개로 한 일을 수만 개에 합니다. 단계마다 정해야 하는 것과 그 이유입니다.

단계	하는 일	왜
정제	중복 리뷰 제거, 한글과 공백만 남기기, 빈 리뷰 제거	같은 리뷰가 여러 번 있으면 그만큼 더 배우게 된다. 한글만 남기면 빈 문자열이 되는 리뷰도 있다
토큰화	형태소로 자르고 불용어 제거	09-01, 09-03
나누기	학습, 검증, 테스트 세 묶음으로	아래 설명
단어 집합	학습 데이터에서 3번 이상 나온 단어만	드문 단어는 종류는 많지만 등장 횟수는 적어, 버려도 잃는 것이 적다
패딩	길이의 분포를 보고 L을 정함	가장 긴 리뷰에 맞추면 대부분의 칸이 패딩이 된다

- 검증 데이터(validation data): 학습하는 **도중에** 성능을 점검하려고 학습 데이터에서 따로 떼어 둔 데이터. 학습에는 쓰지 않습니다.
- 층화 분할(stratify): 나눌 때 긍정과 부정의 비율이 양쪽에서 같게 유지되도록 나누는 것. 한쪽에 부정 리뷰만 몰리는 일을 막습니다.

왜 테스트 데이터가 있는데 검증 데이터가 또 필요할까? 학습을 언제 멈출지, 어떤 설정이 좋은지는 "처음 보는 데이터에서의 성능"을 보고 정해야 합니다. 이것을 테스트 데이터로 하면, 테스트 데이터에 잘 맞는 모델을 고른 셈이 되어 테스트 점수를 더는 믿을 수 없습니다. 그래서 고르는 데 쓰는 데이터(검증)와 마지막에 한 번 재는 데이터(테스트)를 나눕니다.

왜 단어 집합은 학습 데이터로만 만들까? 검증·테스트 데이터는 모델이 본 적 없는 글이어야 합니다. 거기에만 나오는 단어는 실제 서비스에서 처음 보는 단어처럼 <UNK>로 처리되는 것이 맞습니다.

예시 적용

네이버 영화 리뷰(NSMC)는 리뷰 한 줄과 레이블(1 = 긍정, 0 = 부정)로 이루어져 있습니다. 교재는 학습용 15만 개와 테스트용 5만 개를 모두 쓰고 형태소 분석기로 Mecab을 씁니다. 이 자료는 형태소 분석 시간을 줄이려고 학습용 3만 개, 테스트용 5천 개와 Okt를 썼습니다(실습 2).

단계	이 자료 (실습 2)	교재
정제 후 리뷰 수	30,000 → 29,396개	150,000 → 약 146,000개
학습 / 검증 / 테스트	23,516 / 5,880 / 4,923	116,314 / 29,079 / 48,852
서로 다른 단어	18,948개	
2번 이하로 나온 단어	종류의 62.2%, 등장 횟수의 5.8%	종류의 57.6%, 등장 횟수의 2.3%
단어 집합의 크기	7,170	19,193
리뷰 길이	최대 63, 평균 10.7	최대 74, 평균 12.3
길이 30 이하인 리뷰	94.5%	92.5%

드문 단어는 종류로는 절반이 넘지만 등장 횟수로는 5.8%뿐입니다. 이 단어들을 버리면 단어 집합이 18,948개에서 7,170개로 줄어 모델이 훨씬 작아지고, 잃는 것은 적습니다.

네이버 영화 리뷰 (이 자료에서 쓴 개수)

학습 데이터 23,516	검증 데이터 5,880	테스트 데이터 4,923
가중치를 고치는 데 쓴다	에포크마다 점검하고 가장 좋은 모델을 고른다	마지막에 한 번 성능을 잰다

모델은 학습 데이터만 보고 배운다. 검증·테스트 데이터는 한 번도 배우지 않은 '처음 보는 문제'다

그림 2 세 묶음의 크기와 역할. 긍정 리뷰의 비율은 학습 데이터에서 0.497로 거의 반반이다

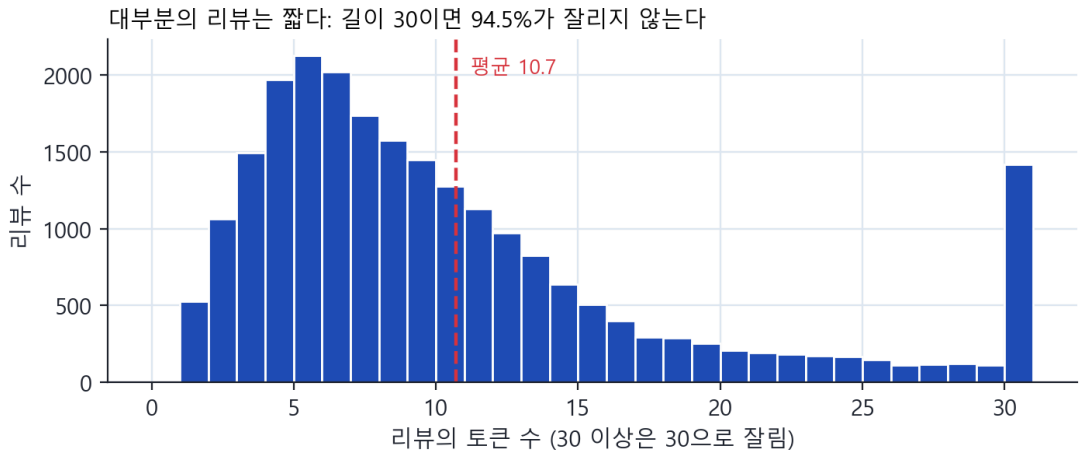


그림 3 학습 리뷰의 길이 분포. 평균은 10.7토큰이고, 길이를 30으로 맞추면 리뷰의 94.5%가 잘리지 않고 들어간다

가장 긴 리뷰(63)에 맞추면 평균 10.7개짜리 리뷰마다 패딩이 50칸 넘게 붙습니다. 30으로 정하면 5.5%의 리뷰만 뒤가 잘리고 텐서는 절반 크기가 됩니다. 첫 리뷰 "아 더빙 진짜 짜증나네요 목소리"는 [아, 더빙, 진짜, 짜증나다, 목소리]로 잘립니다.

2) LSTM 분류 모델

정의

$$(N,\ L) \xrightarrow{\texttt{nn.Embedding}(V,\ D)} (N,\ L,\ D) \xrightarrow{\texttt{nn.LSTM}(D,\ H)} \underbrace{h_L}_{(N,\ H)} \xrightarrow{\texttt{nn.Linear}(H,\ 2)} (N,\ 2)$$

- 13-01의 구조에서 RNN을 LSTM으로 바꾼 것입니다. LSTM은 긴 문장에서 앞 내용을 더 잘 기억합니다(07-02).
- `nn.LSTM` 은 `(전체 출력, (hidden, cell))` 을 돌려줍니다. `hidden` 이 마지막 시점의 은닉 상태이고 모양은 (1, 배치, H)입니다. `squeeze(0)` 으로 맨 앞의 1을 없애 (배치, H)로 만듭니다.
- 출력은 [부정의 점수, 긍정의 점수] 2개이고, 손실은 `CrossEntropyLoss` 입니다.
- 하이퍼파라미터: 임베딩 차원, 은닉 크기, 학습률처럼 사용자가 정해 주는 값. 모델이 학습으로 배우는 가중치와 구분합니다.

예시 적용

임베딩 차원 $D = 100$, 은닉 크기 $H = 128$, 배치 크기 32로 둡니다. 리뷰 32개가 모델을 지날 때의 모양은 이렇습니다(실습 4).

단계	모양
입력	(32, 30)
임베딩	(32, 30, 100)
LSTM의 전체 출력 (쓰지 않음)	(32, 30, 128)
마지막 은닉 상태	(32, 128)
점수	(32, 2)

학습 데이터 23,516개를 32개씩 묶으면 배치가 735개입니다. 가중치는 한 에포크에 735번 고쳐집니다.

3) 학습, 검증, 체크포인트

정의

- 에포크(epoch): 학습 데이터 전체를 한 번 다 쓰는 것. 배치(batch): 한 번에 넣는 묶음. 배치마다 순전파, 손실, 역전파, 가중치 갱신을 합니다(03-04).
- 학습 모드와 평가 모드: `model.train()` 은 학습할 때, `model.eval()` 은 평가할 때 부릅니다. 드롭아웃처럼 학습할 때와 평가할 때 다르게 동작하는 층이 있기 때문입니다.
- `torch.no_grad()` : 평가할 때는 가중치를 고치지 않으므로 기울기를 계산하지 않게 합니다. 빨라지고 메모리를 덜 씁니다.
- 체크포인트(checkpoint): 학습 도중의 모델 가중치를 파일로 저장해 둔 것. 에포크마다 검증 손실을 재고, **지금까지 가운데 가장 낮을 때만** 저장합니다.
- 과적합(overfitting): 학습 데이터에는 점점 잘 맞지만 처음 보는 데이터에는 오히려 나빠지는 현상(06-07). 학습 손실은 줄어드는데 검증 손실이 오르기 시작하면 과적합입니다.

왜 마지막 에포크의 모델이 아니라 검증 손실이 가장 낮은 모델을 쓸까? 학습을 오래 할수록 학습 데이터를 외우게 됩니다. 어느 시점부터는 외우는 만큼 처음 보는 글에서는 나빠집니다. 검증 손실이 가장 낮았던 순간이 "처음 보는 글을 가장 잘 맞히던 순간"이므로 그때의 모델을 씁니다.

예시 적용

5 에포크를 학습한 결과입니다(실습 4).

에포크	학습 손실	학습 정확도	검증 손실	검증 정확도	저장
1	0.6936	0.5017	0.6907	0.5102	저장
2	0.5228	0.7350	0.4389	0.7971	저장
3	0.3622	0.8390	0.4189	0.8138	저장 (가장 낮음)
4	0.2812	0.8784	0.4285	0.8204	
5	0.2098	0.9131	0.4490	0.8180	

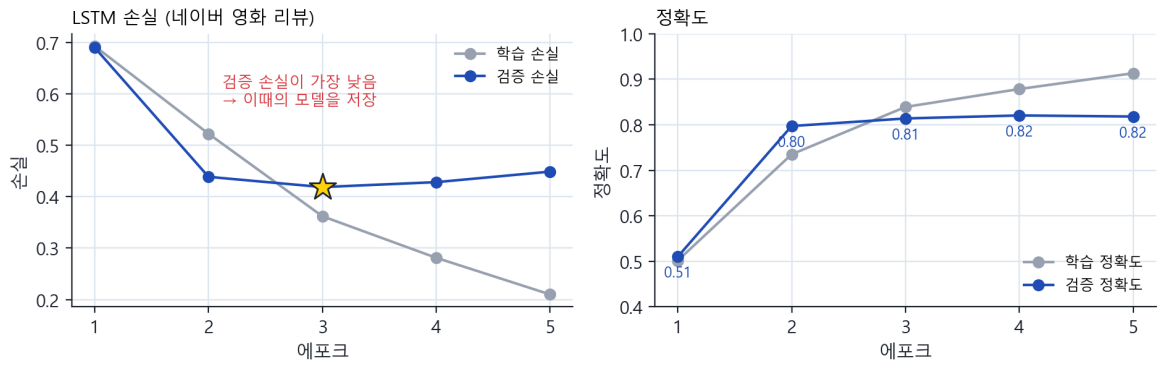


그림 4 학습 손실(회색)은 계속 줄지만 검증 손실(파랑)은 3 에포크에서 가장 낮고 그 뒤로 다시 오른다. 별표가 저장된 모델이다

- **1 에포크**: 손실 0.69는 둘 중 찍을 때의 손실 $\ln 2 = 0.693$과 같습니다. 한 에포크 동안 거의 배우지 못했습니다. 길이를 맞추려고 뒤에 붙인 패딩의 영향으로 보이며, 13-03에서 실험으로 확인합니다.
- **4, 5 에포크**: 학습 정확도는 0.88, 0.91로 계속 오르지만 검증 손실은 0.4285, 0.4490으로 나빠집니다. 과적합이 시작됐습니다.
- 저장된 3 에포크의 모델로 테스트 데이터를 재면 **정확도 0.8129**입니다. 교재는 5배 많은 데이터로 0.8492를 얻었습니다.

새 리뷰를 넣어 봅니다. 예측할 때도 학습 때와 똑같은 전처리(한글만 남기기, 형태소 토큰화, 불용어 제거, 번호로 바꾸기)를 거쳐야 합니다.

리뷰	토큰	예측	긍정일 확률
이 영화 개꿀잼 ㅋㅋㅋ	영화, 개꿀잼, ㅋㅋㅋ	긍정	0.66
이딴게 영화냐 ㅉㅉ	이딴, 영화, 냐, ㅉㅉ	부정	0.04
배우들 연기는 좋은데 스토리가 별로	배우, 연기, 좋다, 스토리, 별로	부정	0.20
재미없지는 않았다	재미없다, 않다	부정 (틀림)	0.22

틀리는 경우. "재미없지는 않았다"는 재미있었다는 뜻인데 부정으로 판단했습니다. 토큰이 [재미없다, 않다]가 되어 부정적인 단어 둘만 남았고, 부정의 부정이 긍정이 된다는 것을 배울 만큼 그런 문장이 데이터에 많지 않았기 때문입니다. "좋은데 … 별로"처럼 앞뒤가 다른 리뷰는 맞혔습니다.

비유

문제집, 모의고사, 수능

문제집으로 공부하고, 모의고사로 실력을 점검하고, 수능은 한 번만 봅니다. 같은 문제집을 너무 여러 번 풀면 답을 외워서 문제집 점수는 오르지만 모의고사 점수는 떨어지기 시작합니다. 그때가 문제집 반복을 멈출 때입니다. 모의고사 점수가 가장 좋았을 때의 상태로 수능을 보는 것이 가장 좋습니다.

문제집	= = 학습 데이터
모의고사	= = 검증 데이터
수능	= = 테스트 데이터
답을 외워 버림	= = 과적합
모의고사가 가장 좋았던 때의 나	= = 체크포인트

실습 2 네이버 영화 리뷰를 정수 텐서로 준비하기

```
# 네이버 영화 리뷰를 정수 텐서로 바꿔 파일로 저장한다. (약 4분)
# 코랩에서는 먼저:  !pip install konlpy
# 교재는 학습 15만 개, 테스트 5만 개와 Mecab을 쓴다.
# 여기서는 형태소 분석 시간을 줄이려고 학습 3만 개, 테스트 5천 개와 Okt를 쓴다.
import urllib.request
from collections import Counter

import numpy as np
import pandas as pd
import torch
from konlpy.tag import Okt
from sklearn.model_selection import train_test_split

# 1) 데이터 내려받기
base = "https://raw.githubusercontent.com/e9t/nsmc/master/"
for name in ["ratings_train.txt", "ratings_test.txt"]:
    urllib.request.urlretrieve(base + name, filename=name)
train_data = pd.read_table("ratings_train.txt")[:30000]
test_data = pd.read_table("ratings_test.txt")[:5000]
print("처음 모양:", train_data.shape, " 열:", list(train_data.columns))

# 2) 정제: 중복 제거 → 한글과 공백만 남기기 → 빈 리뷰 제거
def clean(df):
    df = df.drop_duplicates(subset=["document"]).dropna()
    korean_only = df["document"].str.replace(r"[^ㄱ-ㅎㅏ-ㅣ가-힣 ]", "", regex=True)
    df = df.assign(document=korean_only)
    return df[df["document"].str.strip() ≠ ""]

train_data, test_data = clean(train_data), clean(test_data)
print("정제 후:", len(train_data), "개 /", len(test_data), "개")

# 3) 토큰화: 형태소로 자르고(기본형) 불용어를 뺀다
stopwords = {"도", "는", "다", "의", "가", "이", "은", "한", "에", "하", "고", "을",
             "를", "인", "듯", "과", "와", "네", "들", "지", "임", "게", "하다"}
okt = Okt()
def tokenize(texts):
    return [[w for w in okt.morphs(t, stem=True) if w not in stopwords]
            for t in texts]

X_train, X_test = tokenize(train_data["document"]), tokenize(test_data["document"])
y_train, y_test = train_data["label"].tolist(), test_data["label"].tolist()
print("첫 리뷰:", train_data["document"].iloc[0], "→", X_train[0])

# 4) 학습 데이터의 20%를 검증 데이터로 (긍정·부정 비율 유지)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train, y_train, test_size=0.2, random_state=0, stratify=y_train)
print(f"학습 {len(X_train)} / 검증 {len(X_valid)} / 테스트 {len(X_test)}",
      f" 학습 데이터의 긍정 비율 {np.mean(y_train):.3f}")

# 5) 단어 집합: 3번 이상 나온 단어만 (0 = 패딩, 1 = 모르는 단어)
counts = Counter(w for sent in X_train for w in sent)
kept = [w for w, c in counts.most_common() if c ⩾ 3]
rare_types = 1 - len(kept) / len(counts)
rare_freq = 1 - sum(counts[w] for w in kept) / sum(counts.values())
print(f"서로 다른 단어 {len(counts):,}개.",
      f"2번 이하로 나온 단어가 종류의 {rare_types:.1%},",
      f"전체 등장 횟수의 {rare_freq:.1%}")
word_to_index = {"<PAD>": 0, "<UNK>": 1, **{w: i + 2 for i, w in enumerate(kept)}}
print("단어 집합의 크기:", len(word_to_index),
      " '영화'의 번호:", word_to_index["영화"])

# 6) 정수 인코딩과 패딩 (길이 30)
max_len = 30
def encode(tokenized):
    out = np.zeros((len(tokenized), max_len), dtype=np.int64)
    for i, sent in enumerate(tokenized):
        ids = [word_to_index.get(w, 1) for w in sent][:max_len]
        out[i, :len(ids)] = ids
    return torch.tensor(out)

lengths = [len(s) for s in X_train]
covered = np.mean([n ⩽ max_len for n in lengths]) * 100
print(f"리뷰 길이: 최대 {max(lengths)}, 평균 {np.mean(lengths):.1f}",
      f"/ {max_len} 이하인 리뷰 {covered:.1f}%")

data = {"X_train": encode(X_train), "y_train": torch.tensor(y_train),
        "X_valid": encode(X_valid), "y_valid": torch.tensor(y_valid),
        "X_test": encode(X_test), "y_test": torch.tensor(y_test),
        "word_to_index": word_to_index, "stopwords": stopwords}
```

```
torch.save(data, "nsmc_data.pt")
for name in ["X_train", "X_valid", "X_test"]:
    print(name, tuple(data[name].shape))
print("첫 샘플:", data["X_train"][0].tolist())
```

```
실행 결과
처음 모양: (30000, 3)  열: ['id', 'document', 'label']
정제 후: 29396 개 / 4923 개
첫 리뷰: 아 더빙 진짜 짜증나네요 목소리 → ['아', '더빙', '진짜', '짜증나다', '목소리']
학습 23516 / 검증 5880 / 테스트 4923  학습 데이터의 긍정 비율 0.497
서로 다른 단어 18,948개. 2번 이하로 나온 단어가 종류의 62.2%, 전체 등장 횟수의 5.8%
단어 집합의 크기: 7170  '영화'의 번호: 2
리뷰 길이: 최대 63, 평균 10.7 / 30 이하인 리뷰 94.5%
X_train (23516, 30)
X_valid (5880, 30)
X_test (4923, 30)
첫 샘플: [376, 66, 62, 640, 12, 1149, 877, 57, 10, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

직접 해 보기 `max_len`을 30에서 15로 줄이면 잘리지 않는 리뷰의 비율이 얼마가 되나요? 길이를 줄이면 무엇을 얻고 무엇을 잃을까요?

실습 3 학습과 평가 함수 (13장의 모든 모델이 함께 쓴다)

```
# 13장의 모든 모델이 함께 쓰는 학습·평가 함수. 코랩에서는 셀 맨 위에
# %%writefile train_utils.py  를 붙여 파일로 저장한 뒤, 다른 실습에서 import 한다.
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

criterion = nn.CrossEntropyLoss()


def evaluate(model, X, y, batch_size=256):
    """데이터 전체에 대한 평균 손실과 정확도를 구한다."""
    model.eval()                        # 평가 모드: 드롭아웃 같은 층이 꺼진다
    loss_sum, correct = 0.0, 0
    # 기울기를 계산하지 않는다 (빠르고 메모리 절약)
    with torch.no_grad():
        for bx, by in DataLoader(TensorDataset(X, y), batch_size=batch_size):
            logits = model(bx)
            loss_sum += criterion(logits, by).item() * len(by)
            correct += (logits.argmax(dim=1) == by).sum().item()
    return loss_sum / len(y), correct / len(y)


def fit(model, data, epochs, lr=0.001, batch_size=32, path="best_model.pth"):
    """학습하면서 에포크마다 검증하고,
    검증 손실이 가장 낮을 때의 모델을 저장한다."""
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    loader = DataLoader(TensorDataset(data["X_train"], data["y_train"]),
                        batch_size=batch_size, shuffle=True)
    best_val_loss, history = float("inf"), []
    for epoch in range(1, epochs + 1):
        model.train()                   # 학습 모드
        loss_sum, correct = 0.0, 0
        for bx, by in loader:
            logits = model(bx)                          # 순전파
            loss = criterion(logits, by)                # 손실
            optimizer.zero_grad()
            loss.backward()                             # 역전파
            optimizer.step()                            # 가중치 갱신
            loss_sum += loss.item() * len(by)
            correct += (logits.argmax(dim=1) == by).sum().item()
        n = len(data["y_train"])
        val_loss, val_acc = evaluate(model, data["X_valid"], data["y_valid"])
        history.append((loss_sum / n, correct / n, val_loss, val_acc))
        saved = ""
        if val_loss < best_val_loss:   # 검증 손실이 좋아졌을 때만 저장 (체크포인트)
            best_val_loss, saved = val_loss, "  ← 저장"
            torch.save(model.state_dict(), path)
        print(f"에포크 {epoch}: 학습 손실 {loss_sum / n:.4f}",
              f"정확도 {correct / n:.4f} | 검증 손실 {val_loss:.4f}",
              f"정확도 {val_acc:.4f}{saved}")
    model.load_state_dict(torch.load(path))             # 가장 좋았던 모델로 되돌린다
    return history
```

직접 해 보기 이 파일은 실행해도 출력이 없습니다. 코랩에서는 셀 맨 위에 `%%writefile train_utils.py`를 붙여 파일로 저장하세요. `evaluate`에 `model.eval()`이 없다면 무엇이 달라질까요?

실습 4 LSTM으로 리뷰 분류하기

```
# 실습 2(데이터 준비)와 실습 3(train_utils.py)을 먼저 실행해 둔다
import re

import torch
import torch.nn as nn
from konlpy.tag import Okt
from train_utils import evaluate, fit

torch.manual_seed(0)
data = torch.load("nsmc_data.pt")
word_to_index, stopwords = data["word_to_index"], data["stopwords"]
vocab_size = len(word_to_index)

class TextClassifier(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
```

```
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x, show=False):
        # (배치, 길이) → (배치, 길이, 100)
        embedded = self.embedding(x)
        lstm_out, (hidden, cell) = self.lstm(embedded)  # hidden: (1, 배치, 128)
        # (배치, 128): 마지막 시점의 은닉 상태
        last_hidden = hidden.squeeze(0)
        # (배치, 2): 부정, 긍정의 점수
        logits = self.fc(last_hidden)
        if show:
            for name, t in [("입력", x), ("임베딩", embedded),
                            ("LSTM 전체 출력", lstm_out),
                            ("마지막 은닉 상태", last_hidden), ("점수", logits)]:
                print(f"  {name:10s} {tuple(t.shape)}")
        return logits

model = TextClassifier(vocab_size, embedding_dim=100, hidden_dim=128, output_dim=2)
print("배치 32개가 모델을 지날 때의 모양:")
model(data["X_train"][:32], show=True)

history = fit(model, data, epochs=5, path="best_lstm.pth")
test_loss, test_acc = evaluate(model, data["X_test"], data["y_test"])
print(f"테스트 손실 {test_loss:.4f}, 테스트 정확도 {test_acc:.4f}")
torch.save(history, "lstm_history.pt")

# 새 리뷰에 대해 예측하기: 학습 때와 같은 전처리를 거쳐야 한다
okt = Okt()
def predict(text):
    text = re.sub(r"[^ㄱ-ㅎㅏ-ㅣ가-힣 ]", "", text)
    tokens = [w for w in okt.morphs(text, stem=True) if w not in stopwords]
    ids = torch.tensor([[word_to_index.get(w, 1) for w in tokens]])   # (1, 길이)
    model.eval()
    with torch.no_grad():
        prob = torch.softmax(model(ids), dim=1)[0]
    label = "긍정" if prob[1] > prob[0] else "부정"
    return f"{label} (긍정일 확률 {prob[1]:.2f})  {tokens}"

for text in ["이 영화 개꿀잼 ㅋㅋㅋ", "이딴게 영화냐 ㅉㅉ", "재미없지는 않았다",
             "배우들 연기는 좋은데 스토리가 별로"]:
    print(f"{text} → {predict(text)}")
```

```
실행 결과
배치 32개가 모델을 지날 때의 모양:
  입력          (32, 30)
  임베딩         (32, 30, 100)
  LSTM 전체 출력 (32, 30, 128)
  마지막 은닉 상태  (32, 128)
  점수          (32, 2)
에포크 1: 학습 손실 0.6936 정확도 0.5017 | 검증 손실 0.6907 정확도 0.5102  ← 저장
에포크 2: 학습 손실 0.5228 정확도 0.7350 | 검증 손실 0.4389 정확도 0.7971  ← 저장
에포크 3: 학습 손실 0.3622 정확도 0.8390 | 검증 손실 0.4189 정확도 0.8138  ← 저장
에포크 4: 학습 손실 0.2812 정확도 0.8784 | 검증 손실 0.4285 정확도 0.8204
에포크 5: 학습 손실 0.2098 정확도 0.9131 | 검증 손실 0.4490 정확도 0.8180
테스트 손실 0.4078, 테스트 정확도 0.8129
이 영화 개꿀잼 ㅋㅋㅋ → 긍정 (긍정일 확률 0.66)  ['영화', '개꿀잼', 'ㅋㅋㅋ']
이딴게 영화냐 ㅉㅉ → 부정 (긍정일 확률 0.04)  ['이딴', '영화', '냐', 'ㅉㅉ']
재미없지는 않았다 → 부정 (긍정일 확률 0.22)  ['재미없다', '않다']
배우들 연기는 좋은데 스토리가 별로 → 부정 (긍정일 확률 0.20)  ['배우', '연기', '좋다', '스토리', '별로']
```

직접 해 보기 `predict`에 직접 쓴 리뷰를 넣어 보세요. "최고의 영화", "최악의 영화", "배우는 좋았지만 다시 보고 싶지는 않다"는 각각 어떻게 판단하나요?

확인 문제

13-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 검증 데이터는 가중치를 고치는 데 쓴다. (O / X) 이름 _____

2 학습을 오래 할수록 검증 손실은 항상 줄어든다. (O / X) 이름 _____

3 체크포인트를 저장하는 기준은? 이름 _____

① 학습 손실이 가장 낮을 때 ② 검증 손실이 가장 낮을 때 ③ 마지막 에포크

4 패딩 길이를 가장 긴 리뷰가 아니라 30으로 정한 이유는? 이름 _____

① 대부분의 리뷰가 30 이하라서 패딩을 줄일 수 있어서 ② 30이 규칙이라서 ③ 단어 집합이 30개라서

5 학습 데이터에는 잘 맞지만 처음 보는 데이터에는 나빠지는 현상을 _____이라고 한다. 이름 _____

6 실습에서 저장된 모델의 테스트 정확도는 _____였다. 이름 _____

7 검증 데이터가 왜 필요한지 설명하고, 실습의 에포크 3~5의 숫자로 과적합을 설명해 보세요. 이름 _____

원문: 13-02 LSTM을 이용한 네이버 영화 리뷰 분류 — https://wikidocs.net/217687 (교재의 학습 15만 개·Mecab을 학습 3만 개·Okt로 바꿔 실행했습니다. 그래서 숫자가 교재와 다릅니다)

13-03 GRU를 이용한 IMDB 리뷰 분류하기

핵심 한 줄 **같은 구조에서 LSTM을 GRU로, 데이터를 영어 리뷰로 바꾼 모델이며, 리뷰가 길어 뒤에 패딩이 많이 붙으면 어느 시점의 은닉 상태를 쓰는지가 학습에 큰 영향을 준다.**

왜 배우나 데이터와 RNN의 종류를 바꿔도 뼈대는 그대로라는 것을 확인합니다. 그리고 이번 데이터는 리뷰가 평균 280단어로 깁니다. 길이를 맞추려고 붙인 패딩이 13-02에서 본 "첫 에포크에 거의 배우지 못하는" 문제의 원인이라는 것을 실험으로 확인합니다.

1) IMDB 데이터와 영어 전처리

정의

흐름은 13-02와 같고, 언어가 영어라서 두 단계가 달라집니다.

단계	한국어 (13-02)	영어 (13-03)
정제	한글과 공백만 남김	모두 소문자로 통일 (09-02)
토큰화	형태소 분석기 (Okt, Mecab)	NLTK의 `word_tokenize` (09-01)
나누기, 단어 집합, 정수 인코딩, 패딩	같음	

예시 적용 IMDB 데이터는 영어 영화 리뷰 5만 개이고 긍정과 부정이 25,000개씩입니다. 절반을 테스트용으로 떼고, 나머지의 20%를 검증용으로 떼어 학습 20,000 / 검증 5,000 / 테스트 25,000으로 나눕니다(실습 5).

	네이버 영화 리뷰 (13-02)	IMDB 리뷰 (13-03)
서로 다른 단어	18,948개	98,220개
단어 집합의 크기	7,170	38,443 (2번 이하 59,779개 제거)
리뷰 길이	최대 63, 평균 10.7	최대 2,818, 평균 279.4
패딩 길이	30 (94.5%가 들어감)	500 (87.8%가 들어감)
학습 데이터의 모양	(23516, 30)	(20000, 500)

IMDB 리뷰는 네이버 리뷰보다 26배쯤 깁니다. 교재에 적힌 데이터 주소는 저장소의 폴더 이름이 바뀌어 지금은 열리지 않습니다. 실습 5에는 현재 주소를 적었습니다.

2) GRU 분류 모델과 뒤쪽 패딩

정의

$$(N,\ L) \xrightarrow{\texttt{nn.Embedding}} (N,\ L,\ D) \xrightarrow{\texttt{nn.GRU}(D,\ H)} (N,\ H) \xrightarrow{\texttt{nn.Linear}(H,\ 2)} (N,\ 2)$$

- 13-02에서 `nn.LSTM`을 `nn.GRU`로 바꾼 것입니다. GRU는 셀 상태가 없어서 `(전체 출력, hidden)`만 돌려줍니다(07-02).
- 뒤쪽 패딩(post-padding): 짧은 글의 **뒤에** 0을 붙여 길이를 맞추는 방법. 지금까지 쓴 방법입니다.

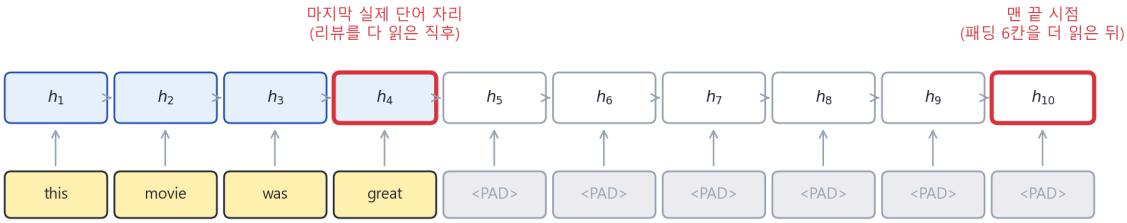
뒤쪽 패딩의 문제. RNN은 패딩도 한 칸씩 읽습니다. 길이 200으로 맞춘 리뷰가 실제로는 50단어라면, 리뷰를 다 읽은 뒤 뜻 없는 패딩 150칸을 더 읽습니다. "맨 끝 시점의 은닉 상태"는 그 뒤의 것이라 리뷰의 내용이 흐려져 있습니다. 07-02의 장기 의존성 문제를 스스로 만든 셈입니다.

해결하는 방법은 몇 가지가 있습니다.

- **마지막 실제 단어 자리의 은닉 상태를 쓴다.** 리뷰마다 패딩이 아닌 토큰의 수를 세고, 전체 출력에서 그 자리의 은닉 상태를 꺼냅니다. 이 자료가 쓴 방법입니다.
- **앞쪽에 패딩한다.** 0을 앞에 붙이면 맨 끝 시점이 곧 마지막 단어입니다.
- 파이토치의 `pack_padded_sequence`로 패딩을 아예 읽지 않게 한다(더 정확하지만 코드가 길어집니다).

교재는 뒤쪽 패딩에 맨 끝 시점의 은닉 상태를 그대로 씁니다. 데이터가 많고 오래 학습하면 모델이 패딩을 지나는 동안 기억을 유지하는 법을 배우기 때문에 결국 학습이 됩니다(교재의 테스트 정확도 0.8641). 다만 그렇게 되기까지 시간이 더 걸립니다.

예시 적용



길이를 맞추려고 뒤에 붙인 패딩도 RNN은 한 칸씩 읽는다. 패딩이 많을수록 맨 끝의 은닉 상태에서 리뷰의 내용이 흐려진다

그림 5 4단어짜리 리뷰를 길이 10으로 맞춘 경우. 빨간 테두리가 두 방법이 각각 쓰는 은닉 상태다

4단어짜리 리뷰를 길이 10으로 맞추면 뒤의 6칸이 패딩입니다. "맨 끝 시점"은 패딩 6칸을 더 읽은 뒤의 은닉 상태 h_{10}이고, "마지막 실제 단어"는 h_4입니다. 코드에서는 리뷰마다 0이 아닌 토큰의 수를 세어(`n_words`) 전체 출력에서 `n_words - 1` 번째 자리를 꺼냅니다.

이 자료의 설정. CPU에서 길이 500짜리를 GRU로 학습하면 시간이 오래 걸려 리뷰의 앞 200토큰만 썼습니다(길이 200 이하인 리뷰는 47.3%이고, 나머지는 뒤가 잘립니다). 마지막 실제 단어 자리의 은닉 상태를 쓰는 모델을 4 에포크 학습한 결과입니다(실습 6).

에포크	학습 손실	학습 정확도	검증 손실	검증 정확도	저장
1	0.6627	0.5919	0.5950	0.6848	저장
2	0.5308	0.7425	0.4889	0.7714	저장
3	0.3623	0.8471	0.3843	0.8360	저장
4	0.2540	0.9027	0.3816	0.8378	저장 (가장 낮음)

테스트 정확도는 0.8272입니다. 교재는 길이 500 전체로 0.8641을 얻었습니다. 리뷰의 뒷부분을 잘라 낸 만큼 낮습니다. 새 리뷰 "This movie was fantastic. I loved every minute of it."은 긍정(0.98), "What a waste of time. The story was really dumb."은 부정(0.01)으로 맞혔고, "It was not bad at all."은 부정(0.08)으로 틀렸습니다. 이 문장은 13-04에서 다시 봅니다.

두 방법의 비교 실험(실습 7). 학습 리뷰 1만 개로 같은 모델을 3 에포크씩, 꺼내는 자리만 바꿔 학습했습니다. 이 1만 개의 평균 단어 수는 171개, 평균 패딩은 29칸입니다.

검증 정확도	1 에포크	2 에포크	3 에포크
맨 끝 시점의 은닉 상태 (교재 방식)	0.5056	0.5976	0.5252
마지막 실제 단어 자리의 은닉 상태	0.6572	0.7280	0.7428

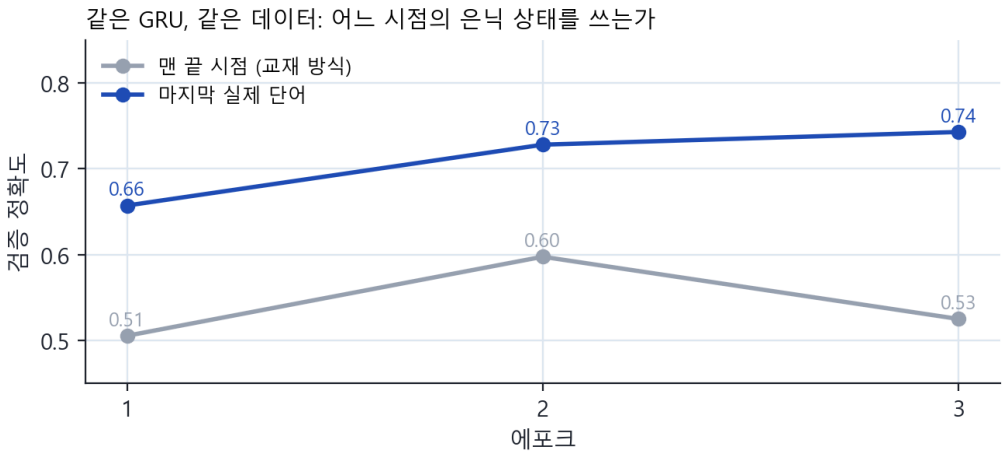


그림 6 같은 모델, 같은 데이터에서 꺼내는 자리만 바꾼 결과. 맨 끝 시점을 쓰면 3 에포크가 지나도 찍는 수준(0.5) 근처에 머문다

- 맨 끝 시점을 쓰면 3 에포크 동안 0.51, 0.60, 0.53으로 오르내리기만 합니다. 마지막 실제 단어를 쓰면 첫 에포크부터 0.66이고 0.74까지 오릅니다.
- 평균 패딩이 29칸뿐인데도 차이가 큽니다. 짧은 리뷰일수록 패딩이 길어서(50단어짜리 리뷰는 150칸), 그런 리뷰들에서 내용이 흐려지기 때문입니다.
- 이 실험은 데이터 1만 개, 3 에포크라는 작은 조건에서 한 것입니다. 교재처럼 더 많은 데이터로 오래 학습하면 맨 끝 시점 방식도 학습이 됩니다. "안 된다"가 아니라 "훨씬 느리다"로 읽어야 합니다.
- 13-02의 LSTM이 첫 에포크에 0.51에 머문 것도 같은 이유로 보입니다. 평균 10.7토큰인 리뷰를 30칸으로 맞췄으니 평균 19칸이 패딩이었습니다.

비유

회의가 끝난 뒤의 잡담

회의 내용을 요약해 달라고 하려면 회의가 끝난 직후에 물어야 합니다. 회의가 끝나고 한 시간 동안 아무 뜻 없는 소리를 더 들은 뒤에 물으면 기억이 흐려져 있습니다. 오래 훈련한 사람은 잡음을 흘려듣고 기억을 지키는 법을 익히지만, 처음부터 그렇게 되지는 않습니다.

회의 내용	= = 리뷰의 실제 단어
뜻 없는 소리	= = 뒤에 붙인 패딩
회의 직후의 기억	= = 마지막 실제 단어 자리의 은닉 상태
한 시간 뒤의 기억	= = 맨 끝 시점의 은닉 상태

실습 5

IMDB 리뷰를 정수 텐서로 준비하기

```python
# IMDB 영화 리뷰(영어) 5만 개를 정수 텐서로 바꿔 파일로 저장한다. (약 3분)
# 코랩에서는 먼저:  import nltk; nltk.download('punkt_tab')
import urllib.request
from collections import Counter

import numpy as np
import pandas as pd
import torch
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split

# 1) 데이터 내려받기 (66MB). 교재에 적힌 주소는 폴더 이름이 바뀌어 아래 주소를 쓴다
url = ("https://raw.githubusercontent.com/ukairia777/pytorch-nlp-tutorial/main/"
       "13.%20RNN%20CNN%20Text%20Classification/dataset/IMDB%20Dataset.csv")
urllib.request.urlretrieve(url, filename="IMDB Dataset.csv")
df = pd.read_csv("IMDB Dataset.csv")
df["sentiment"] = df["sentiment"].replace(["positive", "negative"], [1, 0])
print("전체:", df.shape, " 긍정/부정:", df["sentiment"].value_counts().to_dict())

# 2) 학습 20,000 / 검증 5,000 / 테스트 25,000으로 나누기 (긍정·부정 비율 유지)
X_train, X_test, y_train, y_test = train_test_split(
    df["review"].tolist(), df["sentiment"].tolist(),
    test_size=0.5, random_state=0, stratify=df["sentiment"])
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train, y_train, test_size=0.2, random_state=0, stratify=y_train)

# 3) 토큰화: 소문자로 바꾸고 NLTK로 자른다
def tokenize(texts):
    return [word_tokenize(text.lower()) for text in texts]

tok_train, tok_valid = tokenize(X_train), tokenize(X_valid)
tok_test = tokenize(X_test)
print("첫 리뷰의 앞 12토큰:", tok_train[0][:12])

# 4) 단어 집합: 학습 데이터에서 3번 이상 나온 단어만 (0 = 패딩, 1 = 모르는 단어)
counts = Counter(w for sent in tok_train for w in sent)
kept = [w for w, c in counts.most_common() if c >= 3]
word_to_index = {"<PAD>": 0, "<UNK>": 1, **{w: i + 2 for i, w in enumerate(kept)}}
rare = len(counts) - len(kept)
print(f"서로 다른 단어 {len(counts):,}개 중 2번 이하 {rare:,}개 제거",
      f"→ 단어 집합 {len(word_to_index):,}개")

# 5) 정수 인코딩과 패딩 (길이 500으로 맞추고, 넘치면 자른다)
max_len = 500
def encode(tokenized):
    out = np.zeros((len(tokenized), max_len), dtype=np.int64)
    for i, sent in enumerate(tokenized):
        ids = [word_to_index.get(w, 1) for w in sent][:max_len]
        out[i, :len(ids)] = ids
    return torch.tensor(out)

lengths = [len(s) for s in tok_train]
covered = np.mean([n <= max_len for n in lengths]) * 100
print(f"리뷰 길이: 최대 {max(lengths)}, 평균 {np.mean(lengths):.1f}",
      f"/ {max_len} 이하인 리뷰 {covered:.1f}%")

data = {"X_train": encode(tok_train), "y_train": torch.tensor(y_train),
        "X_valid": encode(tok_valid), "y_valid": torch.tensor(y_valid),
        "X_test": encode(tok_test), "y_test": torch.tensor(y_test),
        "word_to_index": word_to_index}
torch.save(data, "imdb_data.pt")
for name in ["X_train", "X_valid", "X_test"]:
    print(name, tuple(data[name].shape))
print("첫 리뷰의 앞 10개 번호:", data["X_train"][0, :10].tolist())
```

실행 결과

```
전체: (50000, 2)  긍정/부정: {1: 25000, 0: 25000}
첫 리뷰의 앞 12토큰: ['have', 'you', 'ever', ',', 'or', 'do', 'you', 'have', ',', 'a', 'pet', 'who']
서로 다른 단어 98,220개 중 2번 이하 59,779개 제거 → 단어 집합 38,443개
리뷰 길이: 최대 2818, 평균 279.4 / 500 이하인 리뷰 87.8%
X_train (20000, 500)
X_valid (5000, 500)
X_test (25000, 500)
첫 리뷰의 앞 10개 번호: [38, 29, 140, 3, 52, 54, 29, 38, 3, 5]
```

직접 해 보기 단어 집합을 만드는 기준 `c >= 3` 을 `c >= 10` 으로 바꾸면 단어 집합이 몇 개로 줄어드나요?

실습 6 GRU로 IMDB 리뷰 분류하기

```
# 실습 5(IMDB 데이터 준비)와 실습 3(train_utils.py)을 먼저 실행해 둔다
# (CPU로 약 10분)
# 교재는 GPU로 길이 500을 그대로 쓴다. CPU에서는 너무 느려서 앞 200토큰만 쓴다.
import torch
import torch.nn as nn
from nltk.tokenize import word_tokenize
from train_utils import evaluate, fit

torch.manual_seed(0)
full = torch.load("imdb_data.pt")
word_to_index = full["word_to_index"]
vocab_size = len(word_to_index)

max_len = 200
data = {k: (v[:, :max_len] if k.startswith("X") else v) for k, v in full.items()}
lengths = (full["X_train"] != 0).sum(dim=1)
print(f"길이 {max_len} 이하인 리뷰: {(lengths <= max_len).float().mean():.1%}",
      f" 학습 데이터: {tuple(data['X_train'].shape)}")

class GRUClassifier(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim,
                 last_real=True):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.gru = nn.GRU(embedding_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)
        self.last_real = last_real

    def forward(self, x):
        embedded = self.embedding(x)                  # (배치, 200, 100)
        gru_out, hidden = self.gru(embedded)          # gru_out: (배치, 200, 128)
        if self.last_real:
            # 패딩을 뺀 실제 단어 수를 세어,
            # 마지막 "실제 단어" 자리의 은닉 상태를 꺼낸다
            n_words = (x != 0).sum(dim=1).clamp(min=1)
            last = gru_out[torch.arange(len(x)), n_words - 1]    # (배치, 128)
        else:
            last = hidden.squeeze(0)                  # 맨 끝 시점 (패딩까지 읽은 뒤)
        return self.fc(last)                          # (배치, 2)

if __name__ == "__main__":
    model = GRUClassifier(vocab_size, embedding_dim=100, hidden_dim=128,
                          output_dim=2)
    history = fit(model, data, epochs=4, batch_size=64, path="best_gru.pth")
    test_loss, test_acc = evaluate(model, data["X_test"], data["y_test"])
    print(f"테스트 손실 {test_loss:.4f}, 테스트 정확도 {test_acc:.4f}")
    torch.save(history, "gru_history.pt")

    def predict(text):
        ids = [word_to_index.get(w, 1) for w in word_tokenize(text.lower())]
        model.eval()
        with torch.no_grad():
            prob = torch.softmax(model(torch.tensor([ids])), dim=1)[0]
        label = "긍정" if prob[1] > prob[0] else "부정"
        return f"{label} (긍정일 확률 {prob[1]:.2f})"

    for text in ["This movie was fantastic. I loved every minute of it.",
                 "What a waste of time. The story was really dumb.",
                 "It was not bad at all."]:
        print(f"{text} → {predict(text)}")
```

```
실행 결과
길이 200 이하인 리뷰: 47.3%  학습 데이터: (20000, 200)
에포크 1: 학습 손실 0.6627 정확도 0.5919 | 검증 손실 0.5950 정확도 0.6848  ← 저장
에포크 2: 학습 손실 0.5308 정확도 0.7425 | 검증 손실 0.4889 정확도 0.7714  ← 저장
에포크 3: 학습 손실 0.3623 정확도 0.8471 | 검증 손실 0.3843 정확도 0.8360  ← 저장
에포크 4: 학습 손실 0.2540 정확도 0.9027 | 검증 손실 0.3816 정확도 0.8378  ← 저장
테스트 손실 0.3954, 테스트 정확도 0.8272
This movie was fantastic. I loved every minute of it. → 긍정 (긍정일 확률 0.98)
What a waste of time. The story was really dumb. → 부정 (긍정일 확률 0.01)
It was not bad at all. → 부정 (긍정일 확률 0.08)
```

직접 해 보기 `nn.GRU`를 `nn.LSTM`으로 바꾸려면 `forward`의 어느 줄을 고쳐야 하나요? 13-02의 모델과 비교해 보세요.

실습 7 실험: 어느 시점의 은닉 상태를 쓰느냐에 따른 차이

```
# 뒤쪽 패딩 문제를 확인하는 실험 (CPU로 약 8분).
# 실습 6의 GRUClassifier를 그대로 쓰고, 같은 데이터(학습 1만 개)와 같은 설정에서
# "어느 시점의 은닉 상태를 쓰는가"만 바꿔 비교한다.
import importlib

import torch
from train_utils import fit

# 파일 이름에 -가 있어 이렇게 불러온다
gru_lab = importlib.import_module("13-03b_gru")
d = gru_lab.data
data = {"X_train": d["X_train"][:10000], "y_train": d["y_train"][:10000],
        "X_valid": d["X_valid"][:2500], "y_valid": d["y_valid"][:2500]}

n_words = (data["X_train"] != 0).sum(dim=1)
print(f"학습 리뷰 1만 개의 평균 단어 수 {n_words.float().mean():.0f}개,",
      f"평균 패딩 수 {(200 - n_words).float().mean():.0f}칸")

results = {}
modes = [("맨 끝 시점 (교재 방식)", False), ("마지막 실제 단어", True)]
for name, last_real in modes:
```

```
    print(f"\n[{name}]")
    torch.manual_seed(0)
    model = gru_lab.GRUClassifier(gru_lab.vocab_size, 100, 128, 2,
                                  last_real=last_real)
    history = fit(model, data, epochs=3, batch_size=64, path="tmp_gru.pth")
    results[name] = [round(h[3], 4) for h in history]

print("\n에포크별 검증 정확도")
for name, accs in results.items():
    print(f"  {name:16s} {accs}")
torch.save(results, "padding_experiment.pt")
```

```
실행 결과
길이 200 이하인 리뷰: 47.3%  학습 데이터: (20000, 200)
학습 리뷰 1만 개의 평균 단어 수 171개, 평균 패딩 수 29칸

[맨 끝 시점 (교재 방식)]
에포크 1: 학습 손실 0.6964 정확도 0.5078 | 검증 손실 0.6921 정확도 0.5056  ← 저장
에포크 2: 학습 손실 0.6762 정확도 0.5580 | 검증 손실 0.7225 정확도 0.5976
에포크 3: 학습 손실 0.6320 정확도 0.6423 | 검증 손실 0.6803 정확도 0.5252  ← 저장

[마지막 실제 단어]
에포크 1: 학습 손실 0.6795 정확도 0.5607 | 검증 손실 0.6226 정확도 0.6572  ← 저장
에포크 2: 학습 손실 0.5889 정확도 0.6936 | 검증 손실 0.5521 정확도 0.7280  ← 저장
에포크 3: 학습 손실 0.4647 정확도 0.7852 | 검증 손실 0.5193 정확도 0.7428  ← 저장

에포크별 검증 정확도
  맨 끝 시점 (교재 방식)   [0.5056, 0.5976, 0.5252]
  마지막 실제 단어         [0.6572, 0.728, 0.7428]
```

직접 해 보기 시간이 된다면 `epochs=3` 을 6으로 늘려 보세요. "맨 끝 시점" 방식도 결국 따라오나요?

확인 문제

13-03 확인 문제

한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 GRU는 LSTM과 달리 셀 상태를 돌려주지 않는다. (O / X) 이름 _____

2 RNN은 패딩 토큰을 건너뛰고 실제 단어만 읽는다. (O / X) 이름 _____

3 영어 리뷰를 전처리할 때 쓴 토큰화 도구는? 이름 _____

① Okt ② word_tokenize ③ Mecab

4 뒤쪽 패딩에서 "맨 끝 시점의 은닉 상태"가 불리한 이유는? 이름 _____

① 패딩을 읽는 동안 리뷰의 내용이 흐려져서 ② 계산이 더 느려서 ③ 단어 집합이 커져서

5 IMDB 리뷰의 평균 길이는 약 _____토큰이다. 이름 _____

6 짧은 글의 뒤에 0을 붙여 길이를 맞추는 것을 _____ 패딩이라고 한다. 이름 _____

7 뒤쪽 패딩의 문제를 회의 뒤의 잡담에 빗대어 설명하고, 해결 방법을 하나 말해 보세요. 이름 _____

원문: 13-03 GRU를 이용한 IMDB 리뷰 분류하기 — https://wikidocs.net/217083 (CPU에서 돌릴 수 있도록 리뷰의 앞 200토큰만 썼고, 마지막 실제 단어 자리의 은닉 상태를 쓰도록 바꿨습니다. 실습 7의 비교 실험은 이 자료에서 추가했습니다)

13-04 1D CNN을 이용한 IMDB 리뷰 분류

핵심 한 줄

1D 합성곱은 커널이 문장 위를 단어 몇 개씩 훑으며 값을 내는 연산이고, 맥스 풀링으로 커널마다 가장 큰 값 하나만 남겨 문장을 벡터 하나로 요약한다.

왜 배우나

문장을 벡터 하나로 요약하는 방법이 RNN만 있는 것은 아닙니다. 08장에서 이미지에 쓴 합성곱을 문장에도 쓸 수 있습니다. "not good", "waste of time"처럼 **이웃한 단어 몇 개의 조합**이 긍정/부정의 단서가 되는 경우가 많고, 합성곱은 바로 그런 조합을 찾아냅니다. RNN보다 훨씬 빠르기도 합니다.

1) 1D 합성곱

정의

- 문장 행렬: 문장이 임베딩 층을 지난 결과. 모양은 (문장 길이 n, 임베딩 차원 k)이고 한 행이 단어 하나의 벡터입니다. RNN의 입력과 같은 것입니다.
- 2D 합성곱(08장): 커널이 이미지 위를 가로와 세로 두 방향으로 움직입니다.
- 1D 합성곱(1D convolution): 커널의 **너비를 임베딩 차원 k와 같게** 둡니다. 그러면 커널은 가로로 움직일 곳이 없고 **세로(단어 순서) 한 방향으로만** 움직입니다. 그래서 커널의 높이만 말하면 되고, 이것을 커널의 크기라고 합니다.
- 크기가 s인 커널은 한 번에 **이웃한 단어 s개**를 봅니다. 겹친 자리끼리 곱해서 모두 더한 값 하나를 내고(08-01과 같은 계산), 한 단어씩 아래로 내려갑니다.

$$\text{output length} = n - s + 1$$

길이 n인 문장을 크기 s인 커널이 한 칸씩 훑으면 $n - s + 1$개의 값이 나옵니다. 커널 하나가 문장 하나에서 벡터 하나를 만듭니다.

커널의 크기가 뜻하는 것. 크기 2인 커널은 단어 2개 묶음(bigram), 크기 3인 커널은 3개 묶음(trigram)을 봅니다. 10장의 N-gram을 신경망으로 보는 셈입니다. 커널은 가중치이므로 "어떤 단어 묶음에 큰 값을 낼지"를 학습합니다. 한 커널은 "not good" 같은 조합에, 다른 커널은 "really great" 같은 조합에 반응하도록 배웁니다.

파이토치의 `nn.Conv1d(입력 채널, 커널 개수, 커널 크기)` 는 입력을 (배치, 임베딩 차원, 문장 길이) 순서로 받습니다. 임베딩 층의 출력은 (배치, 문장 길이, 임베딩 차원)이므로 `permute(0, 2, 1)` 로 뒤의 두 축을 바꿔 줍니다. 08장에서 이미지의 채널이 앞에 왔던 것처럼, 여기서는 임베딩 차원이 채널 자리에 옵니다.

예시 적용

문장 "wait for the video and do n't rent it"(9토큰)을 임베딩 차원 4로 나타내면 문장 행렬은 (9, 4)입니다. 크기 2인 커널 하나를 직접 움직여 봅니다 (실습 8).

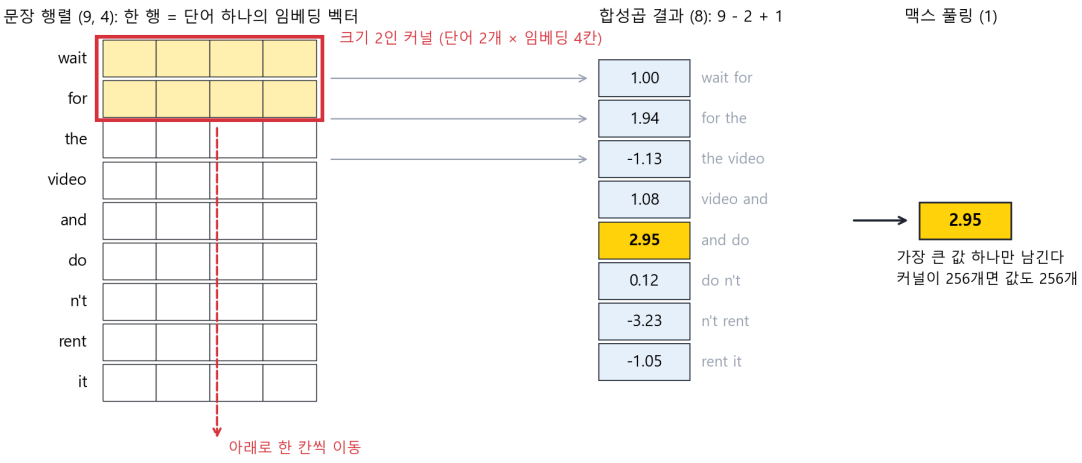


그림 7 크기 2인 커널이 "wait for", "for the", "the video", … 순서로 내려가며 값을 하나씩 낸다. 8개 값 중 가장 큰 것(노랑)이 맥스 풀링의 결과다

- 9 - 2 + 1 = 8번 움직여 값 8개가 나옵니다: 1.00, 1.94, -1.13, 1.08, 2.95, 0.12, -3.23, -1.05
- 이 값을 직접 계산한 것과 `nn.Conv1d` 의 결과가 같습니다(True).
- 커널 크기를 3으로 하면 7개, 5로 하면 5개가 나옵니다.
- 교재의 예: 모양 (32, 16, 50)에 크기 3인 커널 33개를 쓰면 (32, 33, 48)이 됩니다. 길이는 50 - 3 + 1 = 48, 채널 자리는 커널의 개수 33이 됩니다.

여기서 커널은 무작위 값이라 숫자에 뜻은 없습니다. 학습하면 긍정/부정을 가르는 단어 묶음에서 큰 값이 나오도록 커널이 바뀝니다.

2) 맥스 풀링과 CNN 분류 모델

정의

- 맥스 풀링(max-pooling): 커널 하나가 낸 결과 벡터에서 **가장 큰 값 하나**만 남기는 연산. 커널이 F개면 값도 F개가 남습니다.
- 드롭아웃(dropout): 학습할 때 값의 일부를 무작위로 0으로 만들어 과적합을 줄이는 방법(06-07). 평가할 때는 꺼집니다. 그래서 `model.eval()` 이 필요합니다.

$$(N,\ L) \xrightarrow{\text{Embedding}} (N,\ L,\ D) \xrightarrow{\text{permute}} (N,\ D,\ L) \xrightarrow{\text{Conv1d}} (N,\ F,\ L-s+1) \xrightarrow{\max} (N,\ F) \xrightarrow{\text{Linear}} (N,\ 2)$$

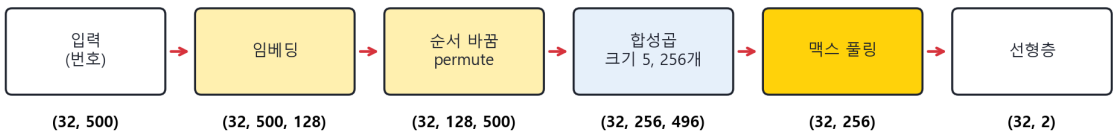
F는 커널의 개수, s는 커널의 크기입니다. 맥스 풀링을 지나면 문장의 길이가 사라지고, 문장마다 F칸짜리 벡터 하나가 남습니다. 이것이 RNN의 마지막 은닉 상태에 해당하는 "문장의 요약"입니다.

왜 가장 큰 값만 남길까? 커널 하나는 "어떤 단어 묶음을 찾는 탐지기"입니다. 값이 크다는 것은 그 묶음을 찾았다는 뜻입니다. 분류에는 "그런 표현이 문장 어딘가에 있었는가"가 중요하고 어디에 있었는지는 덜 중요하므로, 가장 강하게 반응한 값 하나만 남깁니다. 덕분에 리뷰의 길이와 상관없이 같은 크기의 벡터가 됩니다.

	RNN 계열 (LSTM, GRU)	1D CNN
문장을 읽는 방법	앞에서부터 한 단어씩 차례로	모든 자리의 단어 묶음을 한꺼번에
문장의 요약	마지막 은닉 상태 (N, H)	맥스 풀링의 결과 (N, F)
잘 보는 것	단어의 순서와 긴 문맥	이웃한 단어 몇 개의 조합
속도	느림 (시점을 차례로 계산)	빠름 (한꺼번에 계산)
뒤쪽 패딩의 영향	크다 (13-03)	작다 (가장 큰 값만 남기므로)

예시 적용

교재와 같은 설정(임베딩 128차원, 크기 5인 커널 256개, 드롭아웃 0.5)으로 IMDB 리뷰를 학습합니다(실습 9). 이번에는 길이 500을 그대로 씁니다.



길이 500 - 커널 5 + 1 = 496. 맥스 풀링이 길이 방향 496칸을 1칸으로 줄여서, 리뷰의 길이와 상관없이 256칸이 된다

그림 8 리뷰 32개가 CNN을 지날 때의 모양. 합성곱 뒤의 길이는 500 - 5 + 1 = 496이고, 맥스 풀링이 이것을 없앤다

에포크	학습 정확도	검증 손실	검증 정확도
1	0.6416	0.4812	0.7808
2	0.7556	0.3971	0.8268
3	0.8184	0.3151	0.8692

3 에포크 뒤 **테스트 정확도는 0.8559**입니다(교재는 5 에포크에 0.8728). 첫 에포크부터 검증 정확도가 0.78로, 13-02의 LSTM이 첫 에포크에 0.51에 머문 것과 다릅니다. 맥스 풀링은 패딩의 영향을 거의 받지 않기 때문입니다. 학습 정확도가 검증 정확도보다 낮은 것은 드롭아웃이 학습할 때만 켜져 있어서입니다.

리뷰	예측	긍정일 확률
This movie was fantastic. I loved every minute of it.	긍정	0.96
What a waste of time. The story was really dumb.	부정	0.02
It was not bad at all.	부정 (틀림)	0.10

틀리는 경우. "not bad at all"(전혀 나쁘지 않았다)을 부정으로 판단했습니다. 13-02의 "재미없지는 않았다"와 같은 종류의 문장입니다. bad라는 단어에 강하게 반응한 것으로 보이고, "not bad"가 칭찬이라는 것까지는 배우지 못했습니다.

비유

돋보기로 훑으며 형광펜 칠하기

긴 글에서 핵심 표현을 찾을 때, 단어 몇 개가 들어오는 돋보기를 처음부터 끝까지 옮기며 "칭찬하는 표현", "화난 표현"을 찾습니다. 찾는 표현마다 돋보기를 따로 쓰고, 돋보기마다 "가장 강하게 걸린 곳이 얼마나 강했는지"만 적어 둡니다. 그 메모만 보고 글의 분위기를 판단합니다.

돋보기의 폭	= = 커널의 크기
돋보기의 종류 수	= = 커널의 개수
한 칸씩 옮기며 보기	= = 1D 합성곱
가장 강하게 걸린 곳의 세기만 적기	= = 맥스 풀링

실습 8

1D 합성곱과 맥스 풀링을 직접 계산하기

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
sentence = "wait for the video and do n't rent it".split()     # 단어 9개
# 문장 길이 9, 임베딩 차원 4 (보기 쉽게 작게)
n, k = len(sentence), 4
# 문장 행렬 (9, 4): 한 행이 단어 하나의 벡터
matrix = torch.randn(n, k)

# 1) 크기 2인 커널 하나를 직접 움직여 보기: 위에서 아래로 한 칸씩
kernel = torch.randn(2, k)                  # 높이 2(단어 2개), 너비 4(임베딩 차원)
manual = []
for i in range(n - 2 + 1):                  # 9 - 2 + 1 = 8번 움직인다
    window = matrix[i:i + 2]                # 이웃한 단어 2개 (2, 4)
    manual.append((window * kernel).sum()) # 같은 자리끼리 곱해서 모두 더한다
manual = torch.stack(manual)
values = [f"{v:.2f}" for v in manual.tolist()]
print("크기 2 커널의 결과:", tuple(manual.shape), values)
for i in range(3):
    print(f"  {i + 1}번째: '{sentence[i]} {sentence[i + 1]}' → {manual[i]:.2f}")

# 2) nn.Conv1d로 같은 계산: 입력은 (배치, 임베딩 차원, 문장 길이) 순서
conv = nn.Conv1d(in_channels=k, out_channels=1, kernel_size=2, bias=False)
# 같은 커널을 넣는다 (1, 4, 2)
conv.weight.data = kernel.T.unsqueeze(0)
x = matrix.T.unsqueeze(0)                                   # (9, 4) → (1, 4, 9)
print("직접 계산 = nn.Conv1d?", torch.allclose(manual, conv(x)[0, 0], atol=1e-5))

# 3) 커널 크기가 바뀌면 결과 길이가 바뀐다: 문장 길이 - 커널 크기 + 1
for size in [2, 3, 5]:
    out = nn.Conv1d(k, 1, size)(x)
    print(f"  커널 크기 {size}: 결과 {tuple(out.shape)},",
          f"길이 {n} - {size} + 1 = {out.shape[2]}")

# 4) 맥스 풀링: 커널마다 결과 중 가장 큰 값 하나만 남긴다
conv3 = nn.Conv1d(k, 5, kernel_size=3)                      # 크기 3인 커널 5개
out = conv3(x)                                              # (1, 5, 7)
pooled = out.max(dim=2)[0]                                  # (1, 5)
print("커널 5개의 결과:", tuple(out.shape), "→ 맥스 풀링 후:", tuple(pooled.shape))

# 5) 교재의 모양 예시: 배치 32, 임베딩 16, 길이 50에 크기 3인 커널 33개
print("교재 예시:", tuple(nn.Conv1d(16, 33, 3)(torch.randn(32, 16, 50)).shape))
```

실행 결과

```
크기 2 커널의 결과: (8,) ['1.00', '1.94', '-1.13', '1.08', '2.95', '0.12', '-3.23', '-1.05']
  1번째: 'wait for' → 1.00
  2번째: 'for the' → 1.94
  3번째: 'the video' → -1.13
직접 계산 = nn.Conv1d? True
  커널 크기 2: 결과 (1, 1, 8), 길이 9 - 2 + 1 = 8
  커널 크기 3: 결과 (1, 1, 7), 길이 9 - 3 + 1 = 7
  커널 크기 5: 결과 (1, 1, 5), 길이 9 - 5 + 1 = 5
커널 5개의 결과: (1, 5, 7) → 맥스 풀링 후: (1, 5)
교재 예시: (32, 33, 48)
```

직접 해 보기 크기 2인 커널을 크기 4로 바꾸면(`torch.randn(4, k)` , `range(n - 4 + 1)` , `matrix[i:i + 4]`) 결과가 몇 개 나오나요? 식으로 먼저 계산해 보세요.

실습 9

1D CNN으로 IMDB 리뷰 분류하기

```python
# 실습 5(IMDB 데이터 준비)와 실습 3(train_utils.py)을 먼저 실행해 둔다
# (CPU로 약 8분)
import torch
import torch.nn as nn
import torch.nn.functional as F
from nltk.tokenize import word_tokenize
from train_utils import evaluate, fit

torch.manual_seed(0)
data = torch.load("imdb_data.pt")
word_to_index = data["word_to_index"]
vocab_size = len(word_to_index)

class CNN(nn.Module):
    def __init__(self, vocab_size, num_labels):
        super().__init__()
```

```
        self.word_embed = nn.Embedding(vocab_size, 128, padding_idx=0)
        self.conv1 = nn.Conv1d(128, 256, kernel_size=5)   # 크기 5인 커널 256개
        self.dropout = nn.Dropout(0.5)
        self.fc1 = nn.Linear(256, num_labels)

    def forward(self, inputs, show=False):
        e = self.word_embed(inputs)            # (배치, 500, 128)
        # (배치, 128, 500): Conv1d가 받는 순서로
        e = e.permute(0, 2, 1)
        c = self.conv1(e)                      # (배치, 256, 496): 500 - 5 + 1
        p = F.relu(c.max(dim=2)[0])            # (배치, 256): 커널마다 가장 큰 값
        out = self.fc1(self.dropout(p))        # (배치, 2)
        if show:
            for name, t in [("입력", inputs), ("임베딩 후 순서 바꿈", e),
                            ("합성곱", c), ("맥스 풀링", p), ("점수", out)]:
                print(f"  {name:12s} {tuple(t.shape)}")
        return out

model = CNN(vocab_size, num_labels=2)
print("배치 32개가 모델을 지날 때의 모양:")
model(data["X_train"][:32], show=True)

history = fit(model, data, epochs=3, path="best_cnn.pth")
test_loss, test_acc = evaluate(model, data["X_test"], data["y_test"])
print(f"테스트 손실 {test_loss:.4f}, 테스트 정확도 {test_acc:.4f}")
torch.save(history, "cnn_history.pt")

def predict(text):
    ids = [word_to_index.get(w, 1) for w in word_tokenize(text.lower())]
    ids += [0] * max(0, 5 - len(ids))          # 커널 크기 5보다 짧으면 패딩
    model.eval()
    with torch.no_grad():
        prob = torch.softmax(model(torch.tensor([ids])), dim=1)[0]
    return f"{'긍정' if prob[1] > prob[0] else '부정'} (긍정일 확률 {prob[1]:.2f})"

for text in ["This movie was fantastic. I loved every minute of it.",
             "What a waste of time. The story was really dumb.",
             "It was not bad at all."]:
    print(f"{text} → {predict(text)}")
```

실행 결과

```
배치 32개가 모델을 지날 때의 모양:
  입력           (32, 500)
  임베딩 후 순서 바꿈  (32, 128, 500)
  합성곱          (32, 256, 496)
  맥스 풀링        (32, 256)
  점수           (32, 2)
에포크 1: 학습 손실 0.6561 정확도 0.6416 | 검증 손실 0.4812 정확도 0.7808  ← 저장
에포크 2: 학습 손실 0.5025 정확도 0.7556 | 검증 손실 0.3971 정확도 0.8268  ← 저장
에포크 3: 학습 손실 0.3992 정확도 0.8184 | 검증 손실 0.3151 정확도 0.8692  ← 저장
테스트 손실 0.3310, 테스트 정확도 0.8559
This movie was fantastic. I loved every minute of it. → 긍정 (긍정일 확률 0.96)
What a waste of time. The story was really dumb. → 부정 (긍정일 확률 0.02)
It was not bad at all. → 부정 (긍정일 확률 0.10)
```

직접 해 보기 `kernel_size=5` 를 2로 바꾸면 합성곱 뒤의 모양이 어떻게 되나요? "not bad at all"의 예측이 달라지는지도 확인해 보세요.

확인 문제

13-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 1D 합성곱에서 커널은 단어 순서 방향으로만 움직인다. (O / X) 이름 _____

2 맥스 풀링은 결과 벡터의 값을 모두 더한다. (O / X) 이름 _____

3 크기 3인 커널이 한 번에 보는 것은? 이름 _____

① 이웃한 단어 3개 ② 문장 3개 ③ 임베딩의 앞 3칸

4 맥스 풀링을 지나면 무엇이 사라지나요? 이름 _____

① 문장의 길이 ② 커널의 개수 ③ 배치의 크기

5 길이 9인 문장에 크기 2인 커널을 쓰면 결과는 _____개의 값이다. 이름 _____

6 `nn.Conv1d` 는 입력을 (배치, _____ 차원, 문장 길이) 순서로 받는다. 이름 _____

7 1D 합성곱과 맥스 풀링을 정의하고, RNN과 비교해 CNN이 잘 보는 것과 못 보는 것을 설명해 보세요. 이름 _____

원문: 13-04 1D CNN을 이용한 IMDB 리뷰 분류 — https://wikidocs.net/217064 (교재의 5 에포크를 3 에포크로 줄여 실행했습니다)

13-05 사전 훈련된 임베딩을 이용한 성능 상승 시키기

핵심 한 줄

임베딩 층을 무작위 값 대신 사전 훈련된 벡터로 시작하면, 같은 모델과 같은 데이터로도 더 빨리 더 높은 정확도에 도달한다.

왜 배우나

12-07에서 문장 7개짜리 예로 사전 훈련된 임베딩의 효과를 봤습니다. 실제 크기의 문제에서도 그런지 확인합니다. 모델을 바꾸지 않고 시작값만 바꿔 성능을 올리는 것은 실무에서 가장 먼저 시도하는 방법이고, 뒤에 나올 BERT, GPT의 "미리 학습한 것을 가져와 조금 더 학습한다"는 방식의 가장 작은 형태입니다.

1) 사전 훈련된 임베딩으로 임베딩 층 시작하기

정의

13-04의 CNN에서 **임베딩 층만** 바꿉니다. 방법은 12-07과 같습니다.

1. 모양이 (단어 집합의 크기, 사전 훈련된 벡터의 차원)인 임베딩 행렬을 0으로 만듭니다.
2. 내 단어 집합의 단어마다, 사전 훈련된 모델에 있으면 그 벡터를 **그 단어의 번호 행**에 넣습니다.
3. 임베딩 층의 가중치를 이 행렬로 바꾸고, `requires_grad = True` 로 두어 학습하면서 더 고칩니다(미세 조정).

왜 도움이 될까? 임베딩을 처음부터 학습하면 모델은 리뷰 2만 개만 보고 단어의 뜻까지 함께 배워야 합니다. 사전 훈련된 벡터는 수십억 단어에서 배운 뜻을 이미 담고 있습니다. great, wonderful, excellent가 처음부터 가까이 있으므로, 그중 하나에서 배운 것이 나머지에도 바로 적용됩니다. 모델은 "뜻"을 배우는 대신 "이 뜻이 긍정인가"만 배우면 됩니다.

고정할까, 더 고칠까? 12-07에서는 `freeze=True` 로 고정했고, 여기서는 고치도록 둡니다. 데이터가 아주 적으면 고정하는 편이 안전하고(몇 개 안 되는 예로 좋은 벡터를 망가뜨릴 수 있음), 데이터가 충분하면 고치는 편이 문제에 더 잘 맞습니다.

예시 적용

교재는 구글의 Word2Vec(300차원, 3.3GB)을 씁니다. 이 자료는 내려받기 가벼운 GloVe 100차원(128MB)을 썼습니다(실습 10). 그래서 임베딩 차원이 128에서 100으로 바뀌고, 나머지는 13-04와 같습니다.

- 임베딩 행렬의 모양은 (38443, 100)입니다. 단어 집합 38,443개 중 34,664개(90.2%)가 GloVe에 있었고, 나머지 9.8%의 행은 0으로 남습니다.
- apple의 번호는 8067이고, 임베딩 행렬의 8067번 행이 GloVe의 apple 벡터와 같습니다(True). 벡터가 제자리에 들어갔다는 확인입니다.

검증 정확도	1 에포크	2 에포크	3 에포크	테스트 정확도
임베딩을 처음부터 학습 (13-04)	0.7808	0.8268	0.8692	0.8559
사전 훈련된 임베딩으로 시작 (13-05)	0.8742	0.8762	0.8920	0.8908

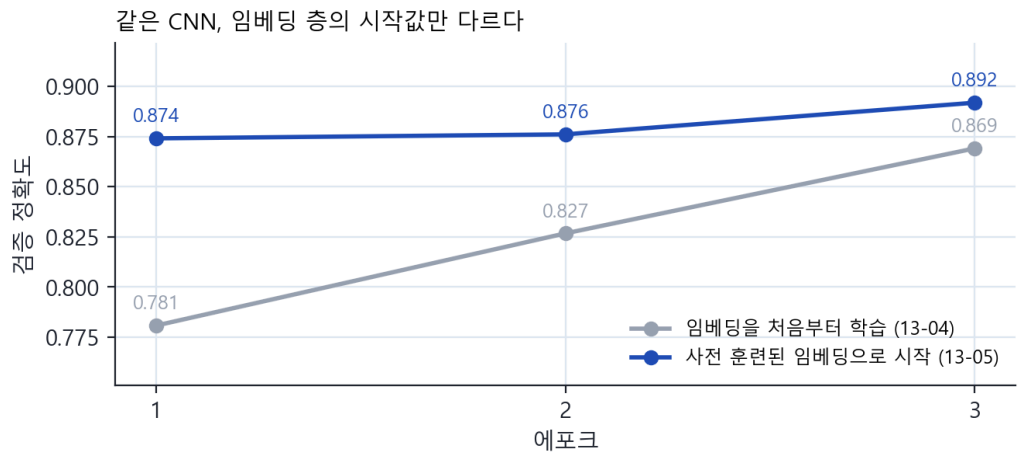


그림 9 같은 CNN, 같은 데이터에서 임베딩의 시작값만 바꾼 결과

- **1 에포크**에 이미 0.8742입니다. 처음부터 학습한 모델이 3 에포크 뒤에 얻은 0.8692보다 높습니다. 출발점이 다르기 때문입니다.
- **테스트 정확도**는 0.8559에서 0.8908로 3.5%p 올랐습니다. 교재도 0.8728에서 0.8983으로 올랐습니다.
- 학습 정확도는 3 에포크에 0.9307까지 올랐고 검증 손실은 0.3058, 0.3063, 0.2822로 움직였습니다. 2 에포크에서는 검증 손실이 좋아지지 않아 저장되지 않았습니다. 더 오래 학습하면 과적합이 시작될 조짐입니다.

비유

영어를 아는 사람과 모르는 사람이 영화 평론을 배울 때

영어를 전혀 모르는 사람에게 영어 리뷰 2만 개로 긍정/부정을 가르치면, 단어의 뜻부터 짐작해 가며 배워야 합니다. 영어를 이미 아는 사람은 같은 2만 개로 "평론에서 어떤 말이 칭찬인가"만 배우면 됩니다. 같은 수업을 들어도 출발점이 달라서 더 빨리, 더 잘 배웁니다.

영어를 모르는 사람	= = 무작위 값으로 시작하는 임베딩
영어를 아는 사람	= = 사전 훈련된 임베딩
평론 수업	= = 리뷰 분류 학습
수업에서 표현을 더 익힘	= = 미세 조정

실습 10

사전 훈련된 임베딩으로 시작하는 CNN

```python
# 실습 5(IMDB 데이터 준비), 실습 3(train_utils.py), 실습 9(13-04의 CNN)를
# 먼저 실행해 둔다 (CPU로 약 8분)
# 교재는 구글 Word2Vec(300차원, 3.3GB)을 쓴다. 여기서는 GloVe 100차원(128MB)을 쓴다.
import gensim.downloader as api
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from train_utils import evaluate, fit

torch.manual_seed(0)
data = torch.load("imdb_data.pt")
word_to_index = data["word_to_index"]
vocab_size = len(word_to_index)

# 1) 사전 훈련된 벡터로 임베딩 행렬 만들기 (12-07과 같은 방법)
glove = api.load("glove-wiki-gigaword-100")
embedding_matrix = np.zeros((vocab_size, 100))
found = 0
for word, i in word_to_index.items():
    # <PAD>, <UNK>와 없는 단어는 0 벡터로 남는다
    if word in glove:
        embedding_matrix[i] = glove[word]
        found += 1
print("임베딩 행렬:", embedding_matrix.shape)
print(f"단어 집합 {vocab_size:,}개 중 GloVe에 있는 단어: {found:,}개",
      f"({found / vocab_size:.1%})")
i = word_to_index["apple"]
same = np.allclose(embedding_matrix[i], glove["apple"])
print("apple의 번호:", i, " 그 행 = GloVe의 apple?", same)

# 2) 13-04의 CNN에서 임베딩 층만 바꾼다
class CNN(nn.Module):
    def __init__(self, vocab_size, num_labels):
        super().__init__()
        self.word_embed = nn.Embedding(vocab_size, 100)
        # 무작위 값 대신 사전 훈련된 값으로 시작하고,
        # 학습하면서 더 고친다(미세 조정)
        pretrained = torch.tensor(embedding_matrix, dtype=torch.float32)
        self.word_embed.weight = nn.Parameter(pretrained)
        self.word_embed.weight.requires_grad = True
        self.conv1 = nn.Conv1d(100, 256, kernel_size=5)
        self.dropout = nn.Dropout(0.5)
        self.fc1 = nn.Linear(256, num_labels)

    def forward(self, inputs):
        e = self.word_embed(inputs).permute(0, 2, 1)
        p = F.relu(self.conv1(e).max(dim=2)[0])
        return self.fc1(self.dropout(p))

model = CNN(vocab_size, num_labels=2)
history = fit(model, data, epochs=3, path="best_cnn_pretrained.pth")
test_loss, test_acc = evaluate(model, data["X_test"], data["y_test"])
print(f"테스트 손실 {test_loss:.4f}, 테스트 정확도 {test_acc:.4f}")
torch.save(history, "cnn_pretrained_history.pt")

# 3) 13-04(임베딩을 처음부터 학습)와 에포크별 검증 정확도 비교
scratch = torch.load("cnn_history.pt")
print("\n에포크 | 처음부터 학습 | 사전 훈련된 임베딩   (검증 정확도)")
for epoch, (a, b) in enumerate(zip(scratch, history), 1):
    print(f"   {epoch}   |    {a[3]:.4f}     |    {b[3]:.4f}")
```

실행 결과

```
임베딩 행렬: (38443, 100)
단어 집합 38,443개 중 GloVe에 있는 단어: 34,664개 (90.2%)
apple의 번호: 8067  그 행 = GloVe의 apple? True
에포크 1: 학습 손실 0.4664 정확도 0.7632 | 검증 손실 0.3058 정확도 0.8742  ← 저장
에포크 2: 학습 손실 0.2802 정확도 0.8828 | 검증 손실 0.3063 정확도 0.8762
에포크 3: 학습 손실 0.1771 정확도 0.9307 | 검증 손실 0.2822 정확도 0.8920  ← 저장
테스트 손실 0.2679, 테스트 정확도 0.8908

에포크 | 처음부터 학습 | 사전 훈련된 임베딩   (검증 정확도)
   1   |    0.7808     |    0.8742
   2   |    0.8268     |    0.8762
   3   |    0.8692     |    0.8920
```

직접 해 보기 `requires_grad = True`를 `False`로 바꿔 임베딩을 고정하면 정확도가 어떻게 달라지나요? 학습 시간은 어떻게 달라지나요?

확인 문제

13-05 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 이 소챕터에서는 13-04의 모델에서 임베딩 층만 바꿨다. (O / X) 이름 _____

2 `requires_grad = True` 이면 가져온 임베딩 벡터는 학습하는 동안 바뀌지 않는다. (O / X) 이름 _____

3 임베딩 행렬에서 단어의 벡터를 넣는 자리는? 이름 _____

① 내 단어 집합에서 그 단어의 번호 행 ② 사전 훈련된 모델에서의 번호 행 ③ 아무 행

4 사전 훈련된 모델에 없는 단어의 행은 어떻게 되나요? 이름 _____

① 0 벡터로 남는다 ② 오류가 난다 ③ 단어 집합에서 지워진다

5 가져온 값에서 시작해 내 데이터로 더 학습하는 것을 _____ 조정이라고 한다. 이름 _____

6 임베딩 행렬의 모양은 (단어 집합의 크기, 사전 훈련된 벡터의 _____)이다. 이름 _____

7 사전 훈련된 임베딩이 왜 도움이 되는지, 영어를 아는 사람과 모르는 사람에 빗대어 설명해 보세요. 이름 _____

원문: 13-05 사전 훈련된 임베딩을 이용한 성능 상승 시키기 — https://wikidocs.net/217099 (교재의 구글 Word2Vec 300차원을 GloVe 100차원으로 바꿔 실행했습니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
텍스트 분류	글을 정해 둔 범주 중 하나로 고르는 지도 학습	스팸/정상, 긍정/부정
다 대 일	마지막 시점의 은닉 상태만 출력층으로	(6, 4, 16) → (6, 16) → (6, 2)
학습 / 검증 / 테스트	배우는 데 / 고르는 데 / 마지막에 한 번 재는 데	23,516 / 5,880 / 4,923
체크포인트, 과적합	검증 손실이 가장 낮을 때의 모델 저장 / 학습 데이터만 잘 맞음	LSTM: 3 에포크에서 저장
뒤쪽 패딩의 문제	패딩을 읽는 동안 맨 끝 은닉 상태에서 내용이 흐려짐	검증 정확도 0.53 vs 0.74 (3 에포크)
1D 합성곱	커널이 이웃한 단어 s개씩 훑음. 결과 길이 $n - s + 1$	9 - 2 + 1 = 8
맥스 풀링	커널마다 가장 큰 값 하나만 남김	(32, 256, 496) → (32, 256)
사전 훈련된 임베딩	임베딩 층을 학습된 벡터로 시작	테스트 0.8559 → 0.8908

오늘 만든 모델의 테스트 정확도. 이 자료는 CPU에서 돌리려고 데이터와 에포크를 줄였으므로 교재보다 조금 낮습니다.

모델	데이터	이 자료	교재
LSTM (13-02)	네이버 영화 리뷰	0.8129 (학습 2.4만 개)	0.8492 (학습 11.6만 개)
GRU (13-03)	IMDB	0.8272 (앞 200토큰)	0.8641 (길이 500)
1D CNN (13-04)	IMDB	0.8559 (3 에포크)	0.8728 (5 에포크)
1D CNN + 사전 훈련된 임베딩 (13-05)	IMDB	0.8908 (GloVe 100차원)	0.8983 (Word2Vec 300차원)

다음

다음 자료 예고: 14장 시퀀스 레이블링 (10/14)

오늘은 문장 전체에 정답 하나를 붙였습니다(다 대 일). 다음에는 문장의 **단어마다** 정답을 붙입니다(다 대 다). 단어마다 품사를 붙이거나, 사람 이름·장소 이름을 찾아내는 문제입니다. 오늘의 모델에서 "마지막 은닉 상태만" 쓰던 것을 "모든 시점의 은닉 상태"를 쓰도록 바꾸면 됩니다.

정답과 해설

13-01 RNN을 이용한 텍스트 분류

1. O
2. X — 외워서 맞힌 것일 수 있습니다. 학습에 쓰지 않은 테스트 데이터로 잽니다.
3. ②
4. ③ — 클래스마다 점수 하나씩
5. 단어 (토큰)
6. 16 — hidden_size
7. 예시 답: 다 대 일은 모든 시점에서 입력을 받지만 마지막에 출력 하나만 내는 구성이다. 평론가가 영화를 끝까지 보면서 인상을 쌓고(은닉 상태) 다 본 뒤에 평가 하나를 내는 것처럼, 마지막 시점의 은닉 상태만 출력층에 넣어 클래스를 고른다.

13-02 LSTM을 이용한 네이버 영화 리뷰 분류

1. X — 학습 도중 성능을 점검하고 모델을 고르는 데만 씁니다.
2. X — 과적합이 시작되면 다시 오릅니다. 실습에서는 4 에포크부터 올랐습니다.
3. ②
4. ① — 94.5%가 30 이하였습니다.
5. 과적합
6. 0.8129 (약 0.81)
7. 예시 답: 학습을 언제 멈출지와 어떤 모델을 쓸지는 처음 보는 데이터에서의 성능으로 정해야 하고, 이것을 테스트 데이터로 하면 테스트 점수를 믿을 수 없게 되므로 검증 데이터를 따로 둔다. 실습에서 학습 손실은 0.36 → 0.28 → 0.21로 계속 줄었지만 검증 손실은 0.4189 → 0.4285 → 0.4490으로 올랐다. 학습 데이터만 더 잘 맞히게 된 과적합이다.

13-03 GRU를 이용한 IMDB 리뷰 분류하기

1. O
2. X — 패딩도 한 칸씩 읽습니다.
3. ②
4. ①
5. 280 (279.4)
6. 뒤쪽
7. 예시 답: 회의가 끝난 뒤 뜻 없는 소리를 한참 더 들으면 회의 내용의 기억이 흐려지듯, RNN이 리뷰를 다 읽은 뒤 패딩을 계속 읽으면 맨 끝 은닉 상태에서 리뷰의 내용이 흐려진다. 패딩이 아닌 마지막 실제 단어 자리의 은닉 상태를 꺼내 쓰거나, 패딩을 앞쪽에 붙이면 된다.

13-04 1D CNN을 이용한 IMDB 리뷰 분류

1. O — 커널의 너비가 임베딩 차원과 같아 가로로는 움직일 곳이 없습니다.
2. X — 가장 큰 값 하나만 남깁니다.
3. ①
4. ① — (32, 256, 496)이 (32, 256)이 됩니다.
5. 8 — 9 - 2 + 1
6. 임베딩
7. 예시 답: 1D 합성곱은 커널이 이웃한 단어 몇 개씩을 훑으며 값을 내는 연산, 맥스 풀링은 커널마다 가장 큰 값 하나만 남기는 연산이다. CNN은 "waste of time" 같은 이웃한 단어의 조합을 빠르게 잘 찾지만, 단어의 순서나 멀리 떨어진 단어 사이의 관계는 RNN보다 잘 보지 못한다.

13-05 사전 훈련된 임베딩을 이용한 성능 상승 시키기

1. O
2. X — 학습 대상이 되어 더 고쳐집니다(미세 조정).
3. ①
4. ①
5. 미세
6. 차원
7. 예시 답: 영어를 모르는 사람은 리뷰를 보며 단어의 뜻부터 배워야 하지만, 영어를 아는 사람은 어떤 말이 칭찬인지만 배우면 된다. 사전 훈련된 임베딩은 단어의 뜻을 이미 담고 있어서, 같은 데이터로도 더 빨리 더 높은 정확도에 도달한다.

용어 정리

용어	정의
텍스트 분류	글을 입력받아 정해 둔 범주 가운데 하나를 고르는 일
지도 학습, 레이블	문제와 정답이 짝지어진 데이터로 학습하는 방법 / 그 정답
학습 / 검증 / 테스트 데이터	가중치를 고치는 데 / 학습 도중 점검하고 모델을 고르는 데 / 마지막에 성능을 재는 데 쓰는 데이터
층화 분할 (stratify)	클래스의 비율이 유지되도록 데이터를 나누는 것
다 대 일	모든 시점에서 입력을 받고 마지막에 출력 하나를 내는 구성
하이퍼파라미터	임베딩 차원, 은닉 크기, 학습률처럼 사용자가 정하는 값
에포크 / 배치	학습 데이터 전체를 한 번 쓰는 것 / 한 번에 넣는 묶음
체크포인트	학습 도중의 모델 가중치를 저장한 것
과적합	학습 데이터에는 잘 맞지만 처음 보는 데이터에는 나빠지는 현상
뒤쪽 패딩	짧은 글의 뒤에 0을 붙여 길이를 맞추는 것
문장 행렬	문장이 임베딩 층을 지난 결과. (문장 길이, 임베딩 차원)
1D 합성곱	커널이 문장 행렬 위를 단어 순서 방향으로만 훑는 합성곱
맥스 풀링	커널마다의 결과에서 가장 큰 값 하나만 남기는 연산
드롭아웃	학습할 때 값의 일부를 0으로 만들어 과적합을 줄이는 방법
미세 조정 (fine-tuning)	가져온 값에서 시작해 내 데이터로 더 학습하는 것